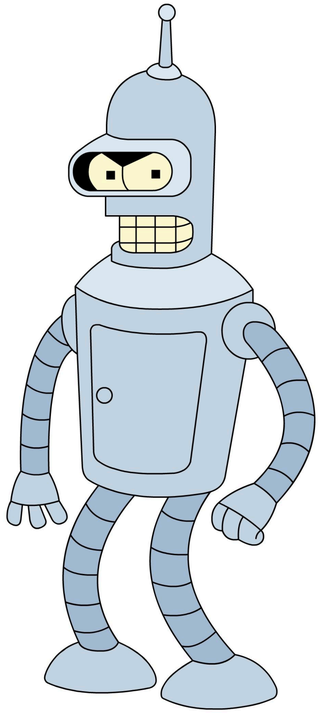
Reinforcement Learning

HSE, winter - spring 2025

Lecture 2: Model-free RL

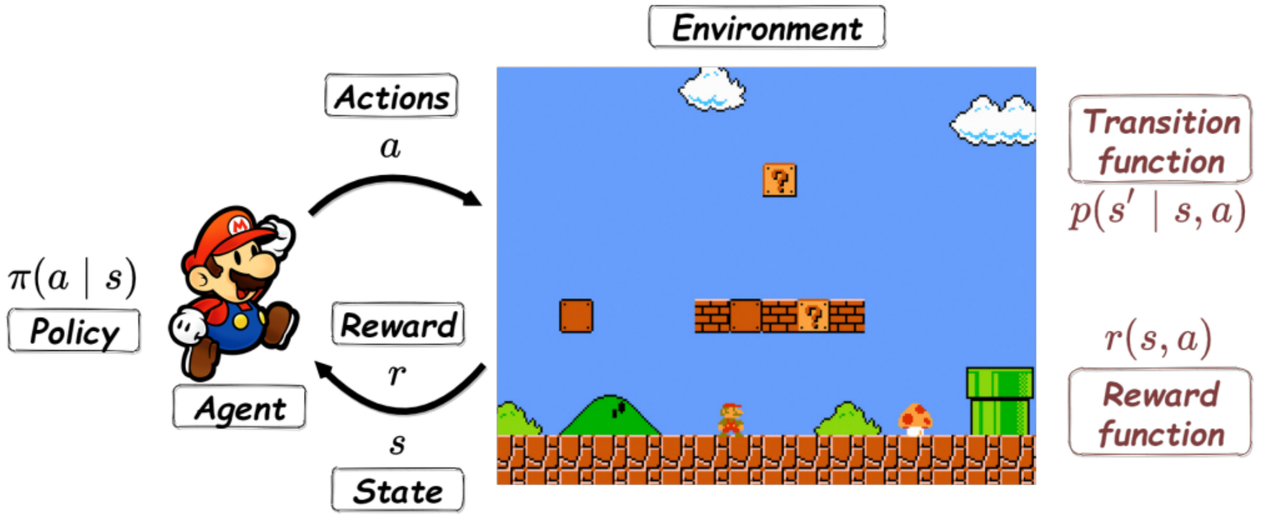


Viacheslav Buchkov

Recap: MDP

MDP is a 4-tuple $(\mathcal{S}, \mathcal{A}, p, r)$:

1. $\mathcal{A}$ is an action space
2. $\mathcal{S}$ is a state space
3. $P : \mathcal{S} \times \mathcal{A} \to \Delta(S)$
   $p(s' | s, a) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a)$
   is a state-transition function
4. $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is a reward function
5. We aim to develop a policy
   $\pi : \mathcal{S} \to \Delta(A)$



Source

$$J(\pi) = \mathbb{E}_\pi[\sum_{t \geq 0} \gamma^t R_t] \to \max_\pi$$

Recap: Value Functions

$$G_t = \sum_{k \geq 0} \gamma^k R_{t+k}$$

$$V^\pi(s) = \mathbb{E}_\pi[G_t \,|\, S_t = s]$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \,|\, S_t = s, A_t = a]$$

Recap: Value Functions

$$G_t = \sum_{k \geq 0} \gamma^k R_{t+k}$$

$V^\pi(s) =$ how much reward one can expect to acquire,
if we start in state s and act with the policy π

$Q^\pi(s, a) =$ how much reward one can expect to acquire,
if we start in state s, take some action a and act with the policy π thereafter

Recap: Value Functions

$Q^{\pi}(s, a) =$ how much reward one can expect to acquire,
if we start in state s, take some action a and act with the policy π thereafter

Recap: Objective

Let T is a final time step. If $T < \infty$ then environment is called *episodic*.

Cumulative reward is called a return or reward-to-go. Note that in general it is a random variable.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \ldots = \sum_{k=t}^{T} \gamma^{k-t} R_k$$

Episode length

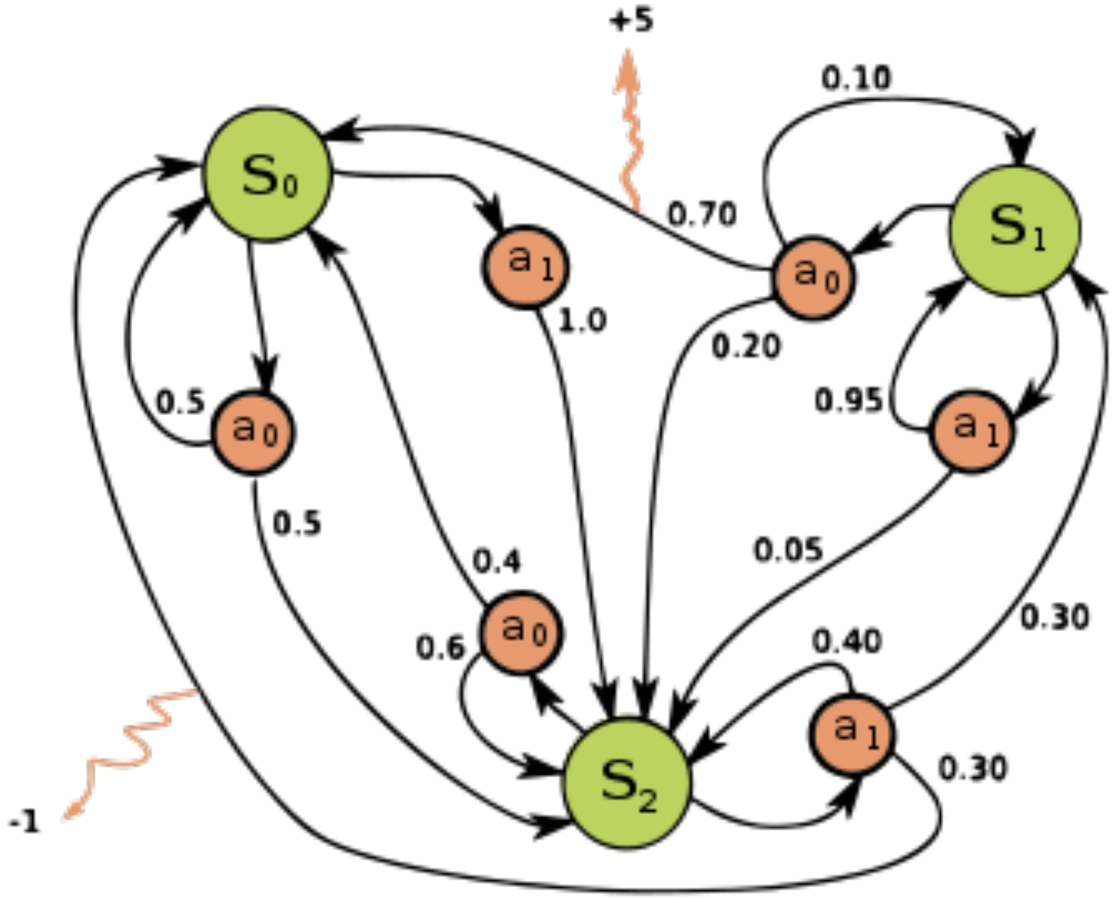
Immediate reward

Discount factor

$$J(\pi) = \mathbb{E}_{\pi}[G_0] \to \max_{\pi}$$

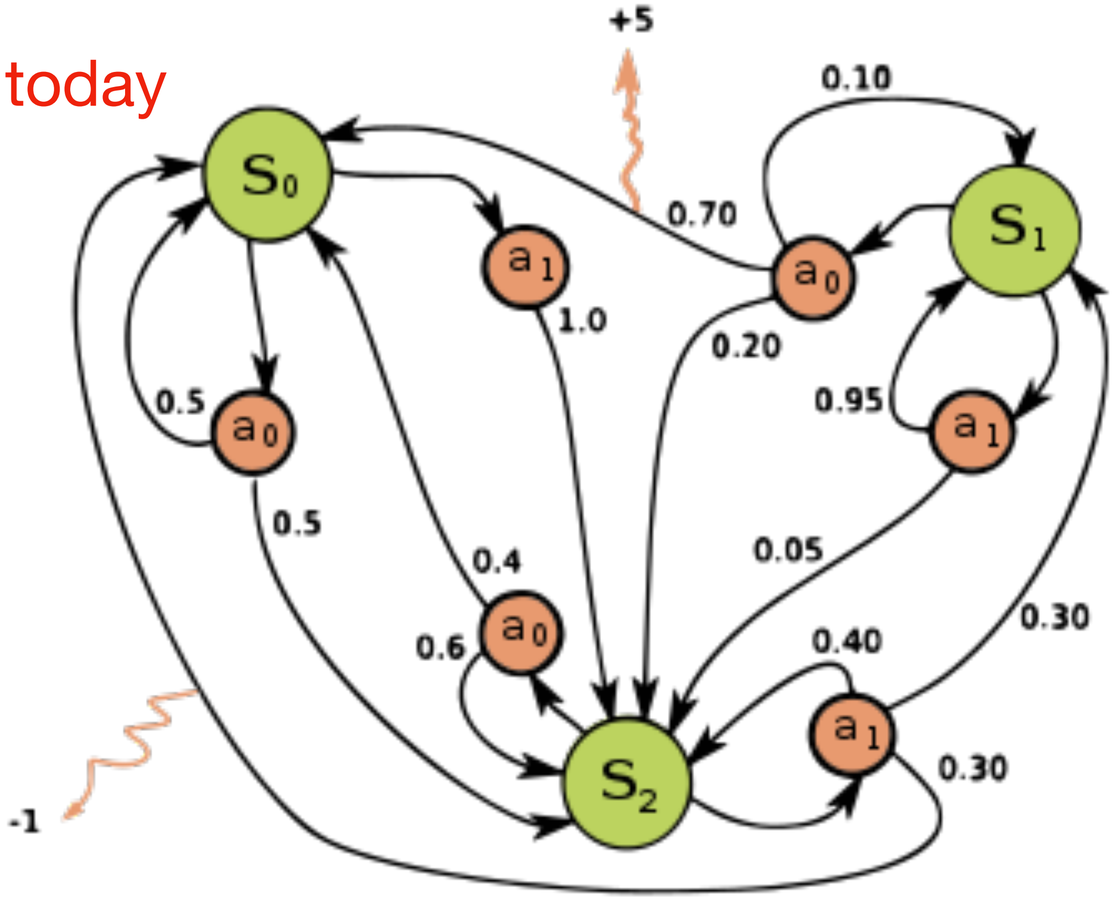
Recap: Assumptions

1. $p(s' \mid s, a)$ is fully known
2. $|\mathcal{S}| < \infty$
3. $|\mathcal{A}| < \infty$



Recap: Assumptions

1. $p(s' \mid s, a)$ is fully known → today
2. $|\mathcal{S}| < \infty$ → Lecture 3
3. $|\mathcal{A}| < \infty$ → Lecture 4



Recap: Bellman Equations

Bellman expectation equations:

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} p(s' \mid s, a) \left[r + \gamma V_{\pi}(s') \right]$$

$$Q^{\pi}(s, a) = \sum_{s'} p(s' \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') Q_{\pi}(s', a') \right]$$

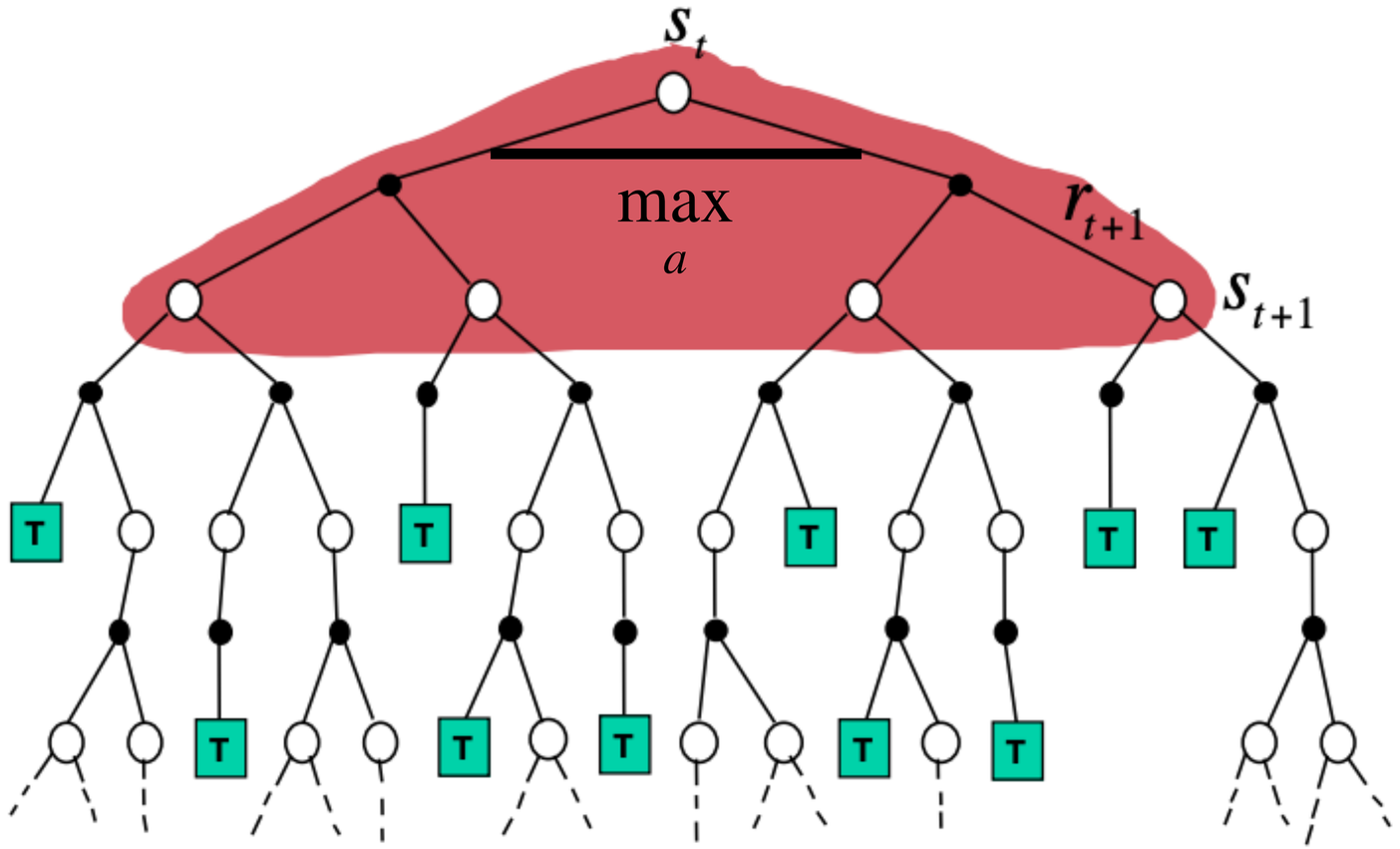
Bellman optimality equations:

$$V^{*}(s) = V^{\pi^{*}}(s) = \max_{a} [r + \gamma \sum_{s'} p(s' \mid s, a) V^{*}(s')]$$

$$Q^{*}(s, a) = \left[r + \gamma \sum_{s'} p(s' \mid s, a) \max_{a'} Q^{*}(s', a') \right]$$

Recap: Value Iteration

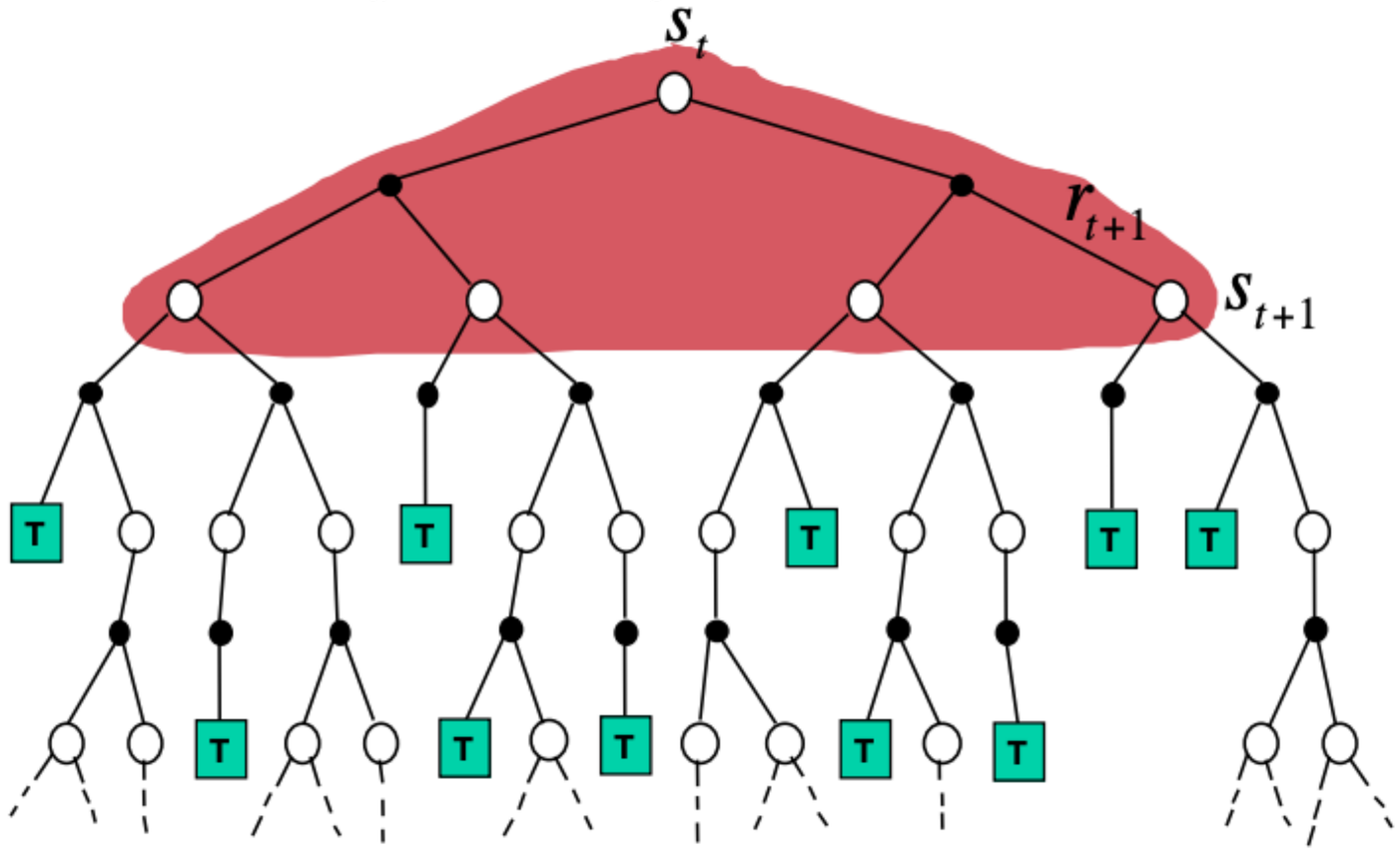
$$V_{k+1}(s) = \max_a \left[r + \gamma \sum_{s'} p(s' \mid s, a) V_k(s') \right]$$



Source

Recap: Policy Evaluation

$$V_{k+1}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} p(s' \mid s, a)\left[r + \gamma V_k(s')\right]$$



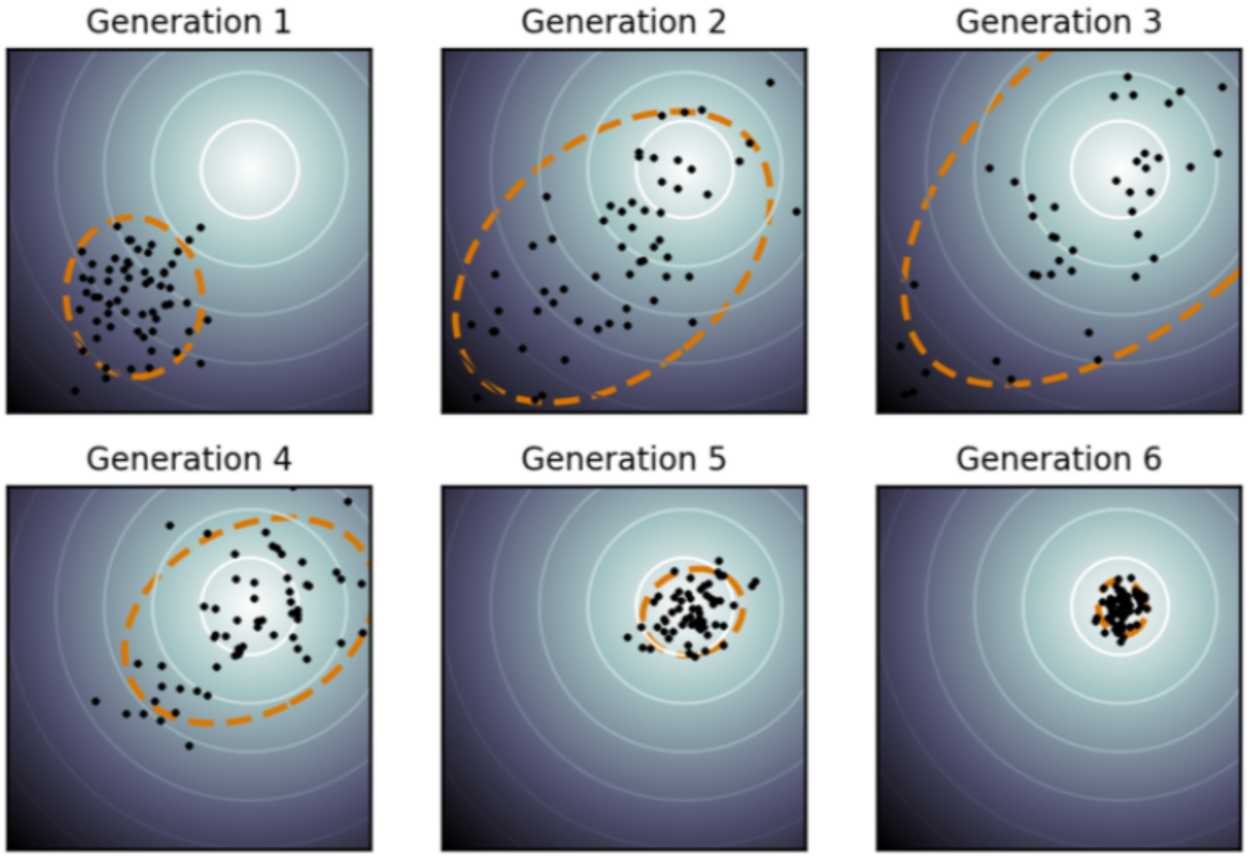
Source

Recap: Policy Improvement

1. If Q_k is known: $\pi(s) = argmax_a Q_k(s, a)$
2. If V_k is known: $\pi(s) = argmax_a \sum_{s'} p(s' \mid s, a)[r + \gamma V_k(s')]$

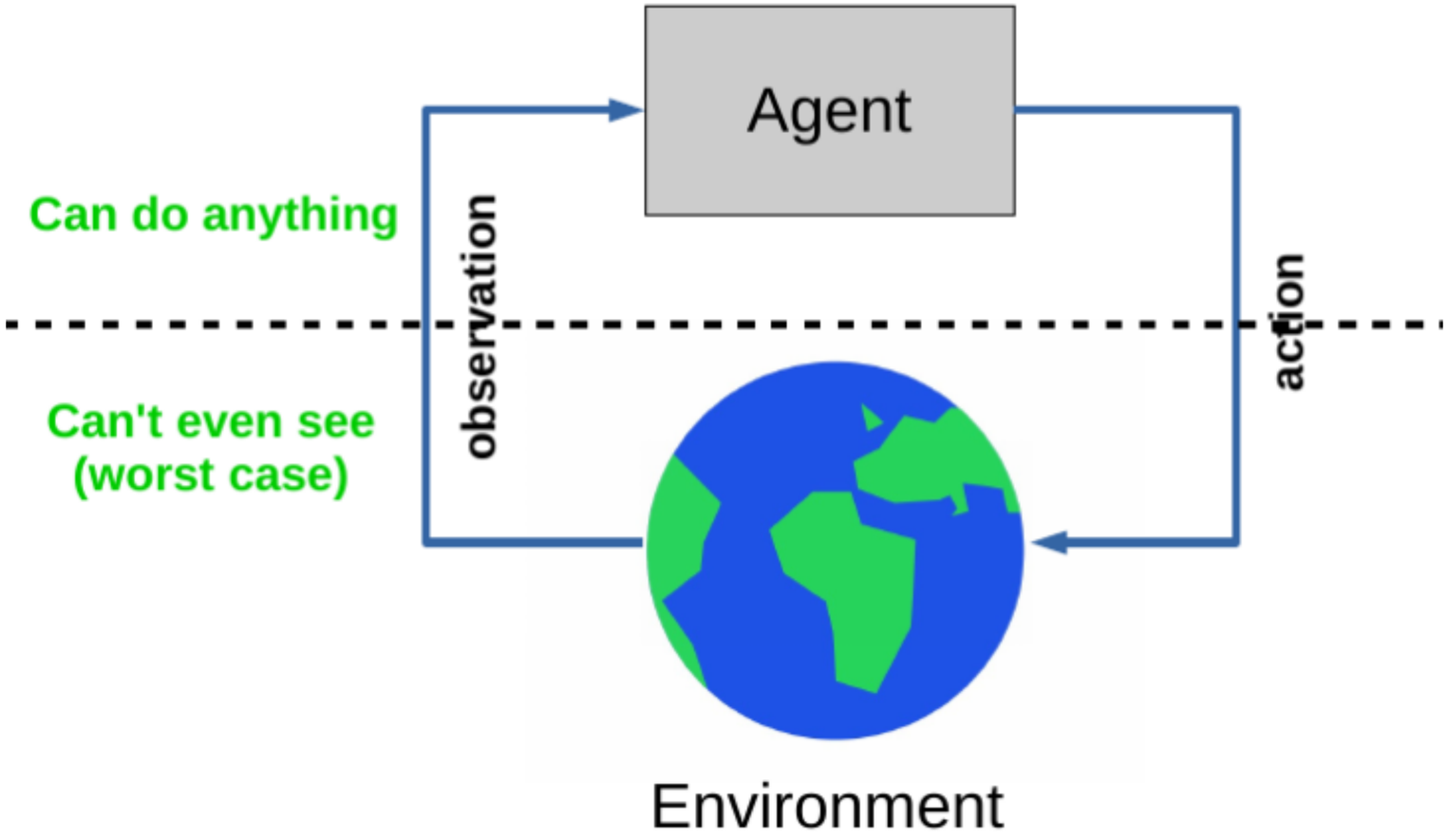
- An optimal policy can be found: $\pi^*(a \mid s) = \mathbb{I}[a = argmax_{a'} Q^*(s, a')]$
- There is always a **deterministic and stationary** optimal policy for any MDP

Recap: Evolution Strategies



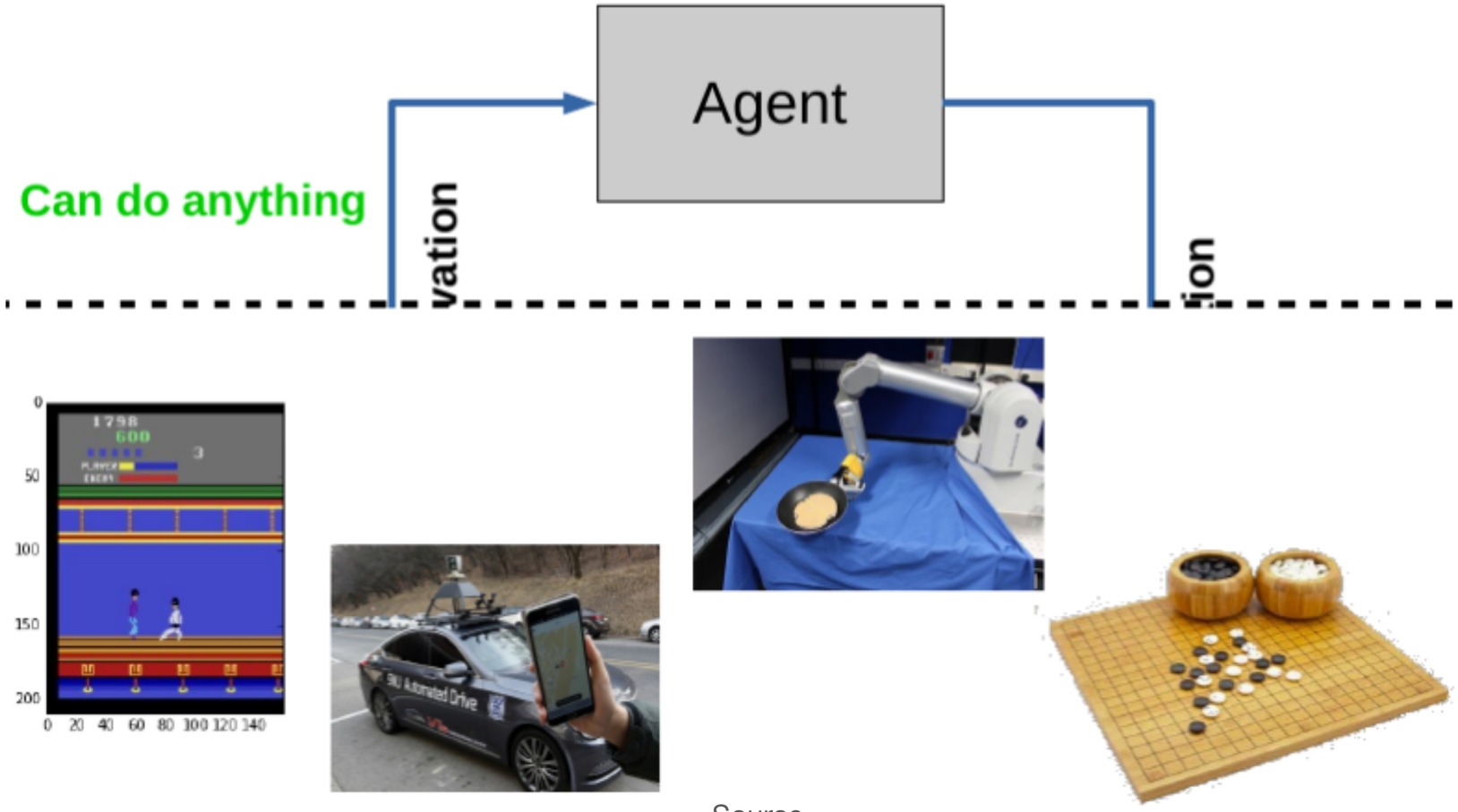
https://lilianweng.github.io/posts/2019-09-05-evolution-strategies/

Decision Processes



Source

Decision Processes



Source

Policy Improvement

$$\pi(s) = \underset{a}{\text{argmax}}\, Q_k(s,a) = \underset{a}{\text{argmax}} \sum_{s'} p(s'|s,a)[r + \gamma V_k(s')]$$

Not available anymore

Policy Improvement

$$\pi(s) = \underset{a}{\text{argmax}}\, Q_k(s,a) = \underset{a}{\text{argmax}} \sum_{s'} p(s'|s,a)[r + \gamma V_k(s')]$$

Not available anymore

If Q_k is known

If V_k is known

Objective Function

- Agent interacts with the environment following some policy $\pi(a \mid s)$.
- Policy stochasticity: $a \sim \pi(\,. \mid s)$
- Environment stochasticity: $s' \sim p(\,. \mid s, a)$
- A trajectory: $\tau = (s_0, a_0, s_1, \ldots, s_{T-1}, a_{T-1}, s_T)$

$$J(\pi) = \mathbb{E}_{\pi}[G_0] = \mathbb{E}_{p(\tau|\pi)}[G_0] =$$

$$= \mathbb{E}_{s_0 \sim p_0}[\mathbb{E}_{a_0 \sim \pi(.|s_0)}[\mathbb{E}_{s_1 \sim p(.|s_0,a_0)}[r_0 + \gamma[\ldots]]]] =$$

$$= \mathbb{E}_{s_0}[\mathbb{E}_{a_0}[\mathbb{E}_{s_1}[r_0 + \gamma[\ldots]]]] \to \max_{\pi}$$

Learn the underlying MDP

Idea: Use observed state-transitions (s, a, r, s') to learn the underlying MDP

Two basic approaches to RL

Idea: Use observed state-transitions (s, a, r, s') to learn the underlying MDP

1. **Model-Based RL**:
   - Learn the MDP:

     Estimate transition probabilities from data $P(s' \mid s, a) = \frac{Count(s', s, a)}{Count(s, a)}$

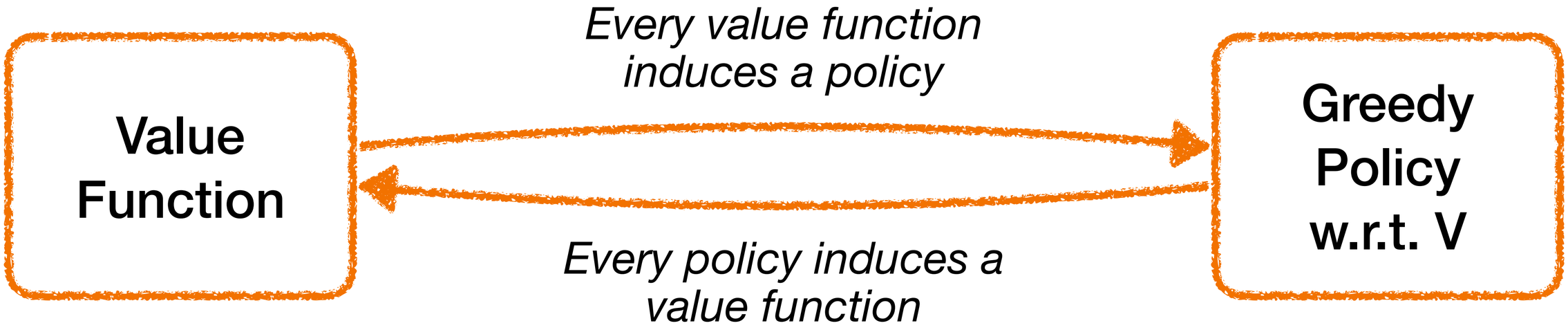
     Estimate reward function from data $r(s, a) = \frac{1}{N(s, a)} \sum_{t:S=s,A=a} R_t$
   - Optimize the policy, based on this MDP (Policy Iteration / Value Iteration)

Two basic approaches to RL

Idea: Use observed state-transitions (s, a, r, s') to learn the underlying MDP

1. **Model-Based RL**:
    - Memory Required: $\mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$
    - Computational Time: need to recompute π_G with VI/PI $\Longrightarrow$ expensive
    - **But more sample efficient**

Value Function and Policy



Theorem (Bellman):

Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$\pi^*(s) \in \operatorname*{argmax}_a [r(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a) V^*(s')] = \operatorname*{argmax}_a Q^*(s,a)$$

Two basic approaches to RL

Idea: Use observed state-transitions (s, a, r, s') to learn the underlying MDP

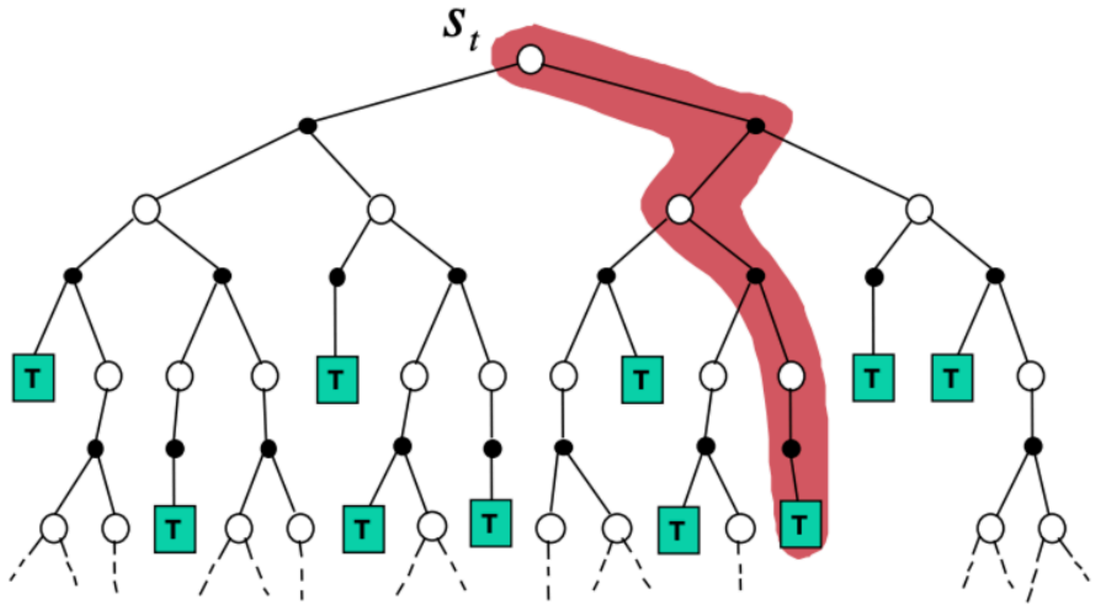
1. **Model-Based RL**:
   - Learn the MDP:

     Estimate transition probabilities from data $P(s' \mid s, a) = \frac{Count(s', s, a)}{Count(s, a)}$

     Estimate reward function from data $r(s, a) = \frac{1}{N(s, a)} \sum_{t:S=s,A=a} R_t$
   - Optimize the policy, based on this MDP (Policy Iteration / Value Iteration)
2. **Model-Free RL**:
   - Estimate the $\hat{Q}(s, a)$ function directly
   - Find a greedy policy π_G, induced by this action-value function

Monte-Carlo Policy Evaluation

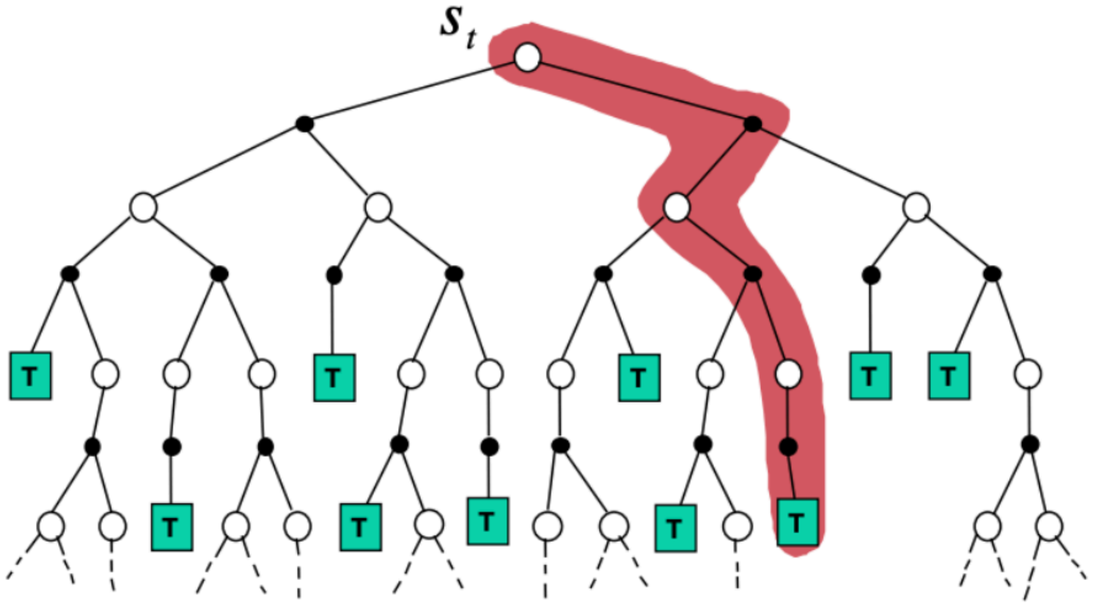
$$\tau_k = \{s_{k0} = s, a_{k1} = a, s_{k1}, a_{k1}, \dots\}, G(\tau_k) = \sum_{t=0}^{T} \gamma^t r_{ko} \implies Q^{\pi_t} \approx \frac{1}{N} \sum_{k=0}^{N} G(\tau_k)$$



Monte-Carlo Policy Evaluation

$$\tau_k = \{s_{k,0} = s, a_{k,0} = a, s_{k,1}, a_{k,1}, \dots\}, G(\tau_k) = \sum_{t=0}^{T} \gamma^t r_{k,t} \implies Q^{\pi_t} \approx \frac{1}{N} \sum_{k=1}^{N} G(\tau_k)$$

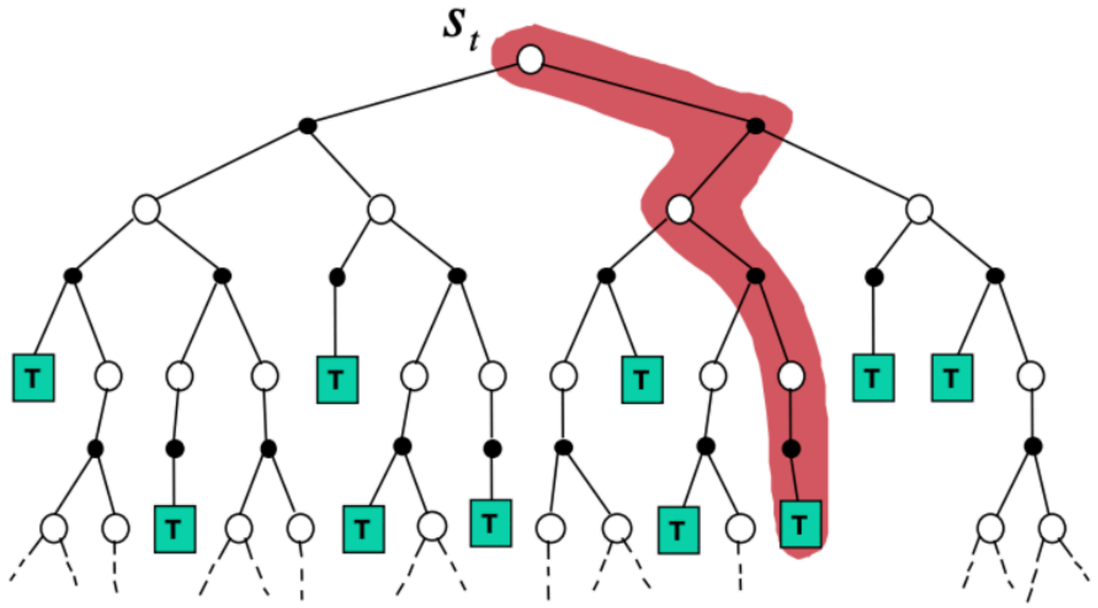
$$Q(s,a) \approx \frac{1}{N} \sum_{k=1}^{N} G(\tau_k) = \frac{1}{N} \sum_{k=1}^{N-1} G(\tau_k) + \frac{1}{N} G(\tau_N)$$



Monte-Carlo Policy Evaluation

$$\tau_k = \{s_{k0} = s, a_{k1} = a, s_{k1}, a_{k1}, \dots\}, G(\tau_k) = \sum_{t=0}^{T} \gamma^t r_{ko} \implies Q^{\pi_t} \approx \frac{1}{N} \sum_{k=0}^{N} G(\tau_k)$$

$$Q(s,a) \approx \frac{1}{N} \sum_{k=1}^{N} G(\tau_k) = \frac{N-1}{N} \frac{1}{N-1} \sum_{k=1}^{N-1} G(\tau_k) + \frac{1}{N} G(\tau_N) = \frac{N-1}{N} \hat{Q}^{t-1}(s,a) + \frac{1}{N} G(\tau_N)$$



Pros:

- Unbiased
- Guarantees on convergence

Cons:

- High variance
- Must complete the episodes

Bellman Equations

Bellman expectation equations:

$$V^{\pi}(s) = \mathbb{E}_{a,s'}[r(s,a) + \gamma V^{\pi}(s')]$$

$$Q^{\pi}(s,a) = \mathbb{E}_{s',a'}\left[r + \gamma Q^{\pi}(s',a')\right]$$

Bellman optimality equations:

$$V^{*}(s) = \max_{a} \mathbb{E}_{s'}[r + \gamma V^{*}(s')]$$

$$Q^{*}(s,a) = \mathbb{E}_{s'}\left[r + \gamma \max_{a'} Q^{*}(s',a')\right]$$

Stochastic Approximation

https://en.wikipedia.org/wiki/Stochastic_approximation

- We would like to estimate $\theta^* = \mathbb{E}[X]$
- Replace it with the following iterative procedure:

$$\theta_{k+1} = (1 - \alpha_k)\theta_k + \alpha_k X_k$$

Stochastic Approximation

https://en.wikipedia.org/wiki/Stochastic_approximation

- We would like to estimate $\theta^* = \mathbb{E}[X]$
- Replace it with the following iterative procedure:

$$\theta_{k+1} = (1 - \alpha_k)\theta_k + \alpha_k X_k = \theta_k - \alpha_k[\theta_k - X_k]$$

Stochastic Approximation

https://en.wikipedia.org/wiki/Stochastic_approximation

- We would like to estimate $\theta^* = \mathbb{E}[X]$
- Replace it with the following iterative procedure:

$$\theta_{k+1} = (1 - \alpha_k)\theta_k + \alpha_k X_k = \theta_k - \alpha_k[\theta_k - X_k]$$

Robbins–Monro theorem:

- $\sum_{k=0}^{+\infty} \alpha_k = +\infty, \sum_{k=0}^{+\infty} \alpha_k^2 < +\infty$ ⟶ $\theta_k \to \theta^*$ in squared mean
- Some technical conditions

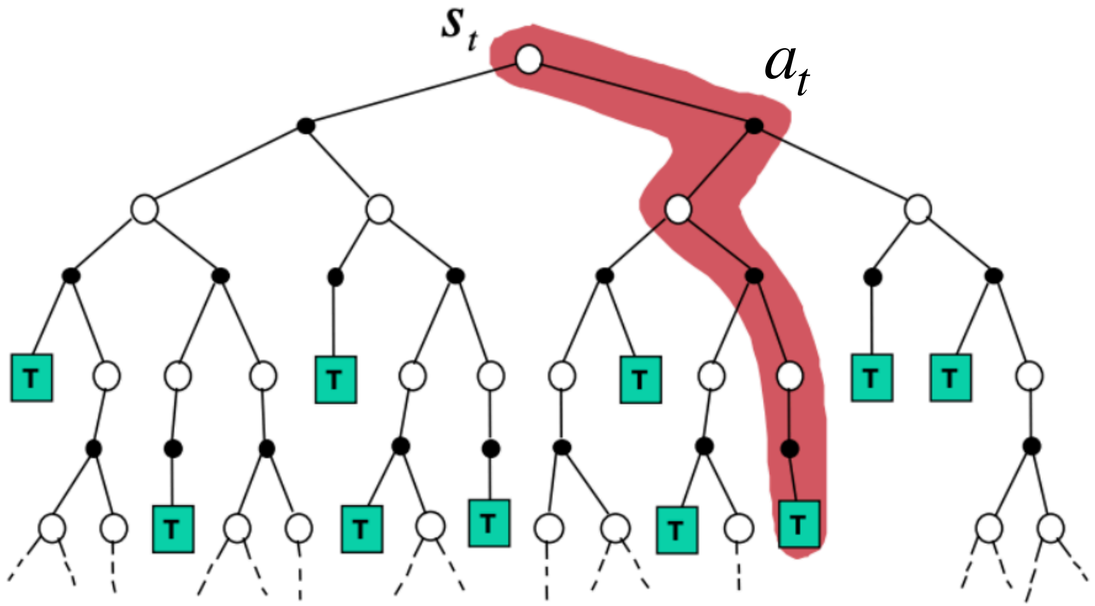
Bellman Equations

Bellman optimality equations:

$$Q^*(s,a) = \mathbb{E}_{s'}\left[r + \gamma \max_{a'} Q^*(s',a')\right]$$

Monte-Carlo Policy Evaluation

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(G_k - Q_k(s,a))$$



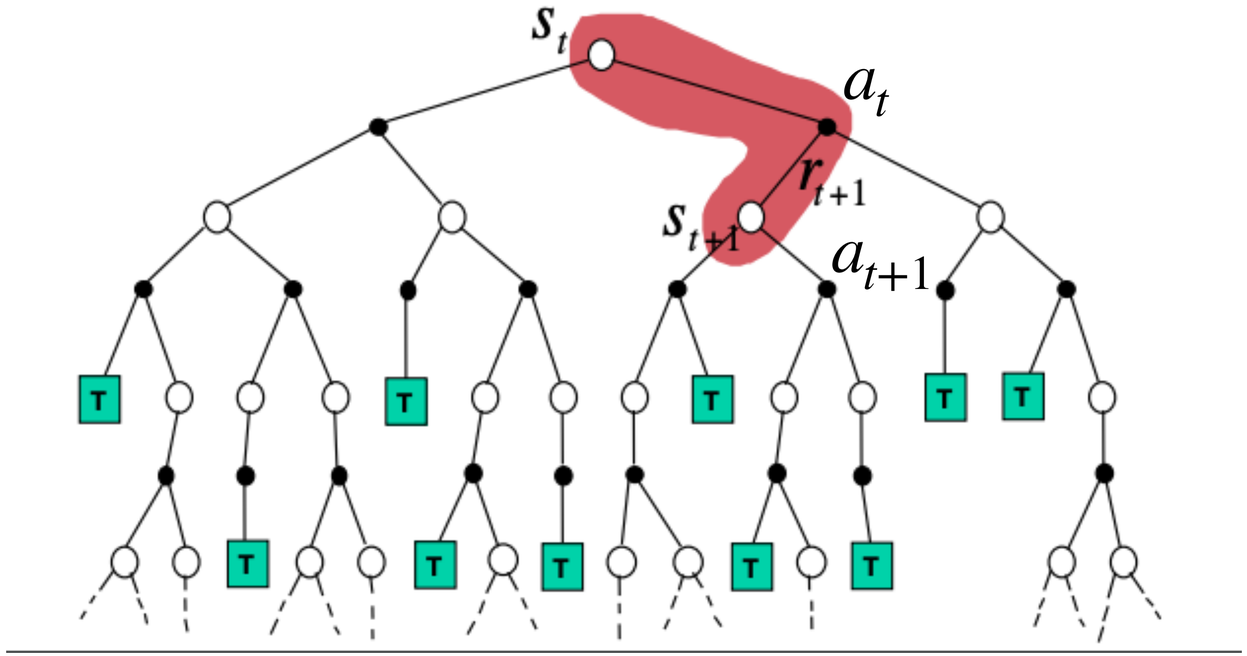
Temporal Difference Learning

Target

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)\big(r + \gamma Q_k(s',a') - Q_k(s,a)\big)$$

Temporal difference

We require: $s' \sim p(\,.\,|\,s,a), a' \sim \pi(\,.\,|\,s)$



Temporal Difference Learning: Example

Target

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(r + \gamma Q_k(s',a') - Q_k(s,a))$$

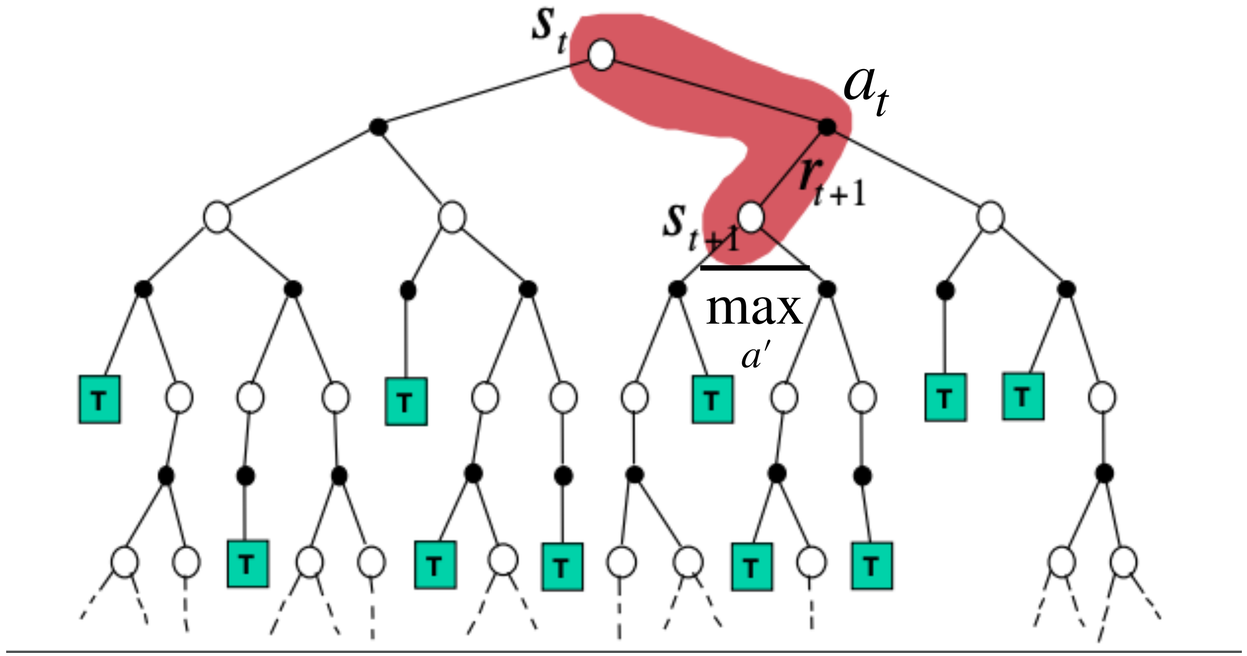
Temporal difference

We require: $s' \sim p(\,.\,|\,s,a), a' \sim \pi(\,.\,|\,s)$

Temporal Difference Learning

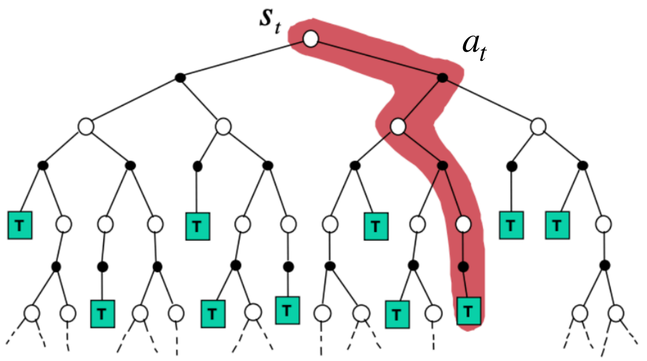
$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$$

We require: $s' \sim p(\,.\,|\,s,a)$

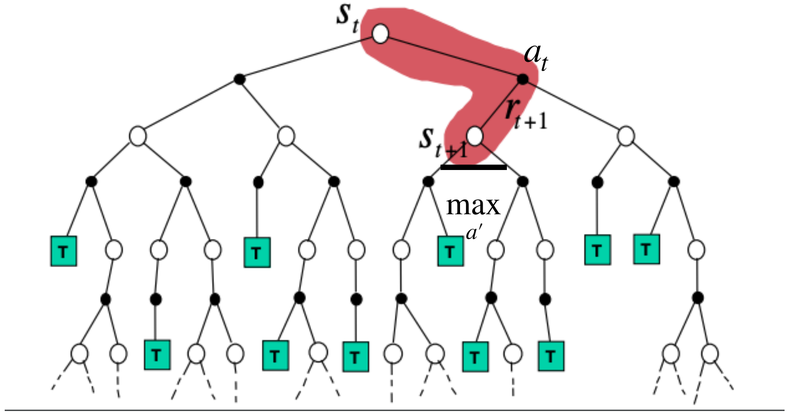


Policy Evaluation

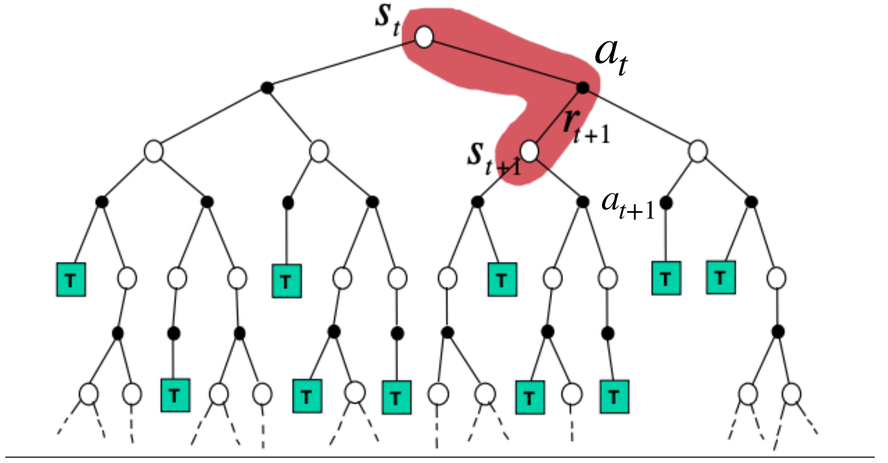
$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(G_k - Q_k(s,a))$$



$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$$



$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(r + \gamma Q_k(s',a') - Q_k(s,a))$$



Temporal Difference Learning

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(y_Q - Q_k(s,a))$$

Infinite visitation:

$$\sum_{k \geq 0} \alpha_k(s,a) = +\infty$$

For example:

$$\alpha_k(s,a) = \frac{1}{n(s,a)}$$

Stochastic policy $\mu(a \mid s)$ s.t.

$\forall s, a : \mu(a \mid s) > 0$:

Temporal Difference Learning

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(y_Q - Q_k(s,a))$$

Infinite visitation:

$$\sum_{k \geq 0} \alpha_k(s,a) = +\infty$$

For example:

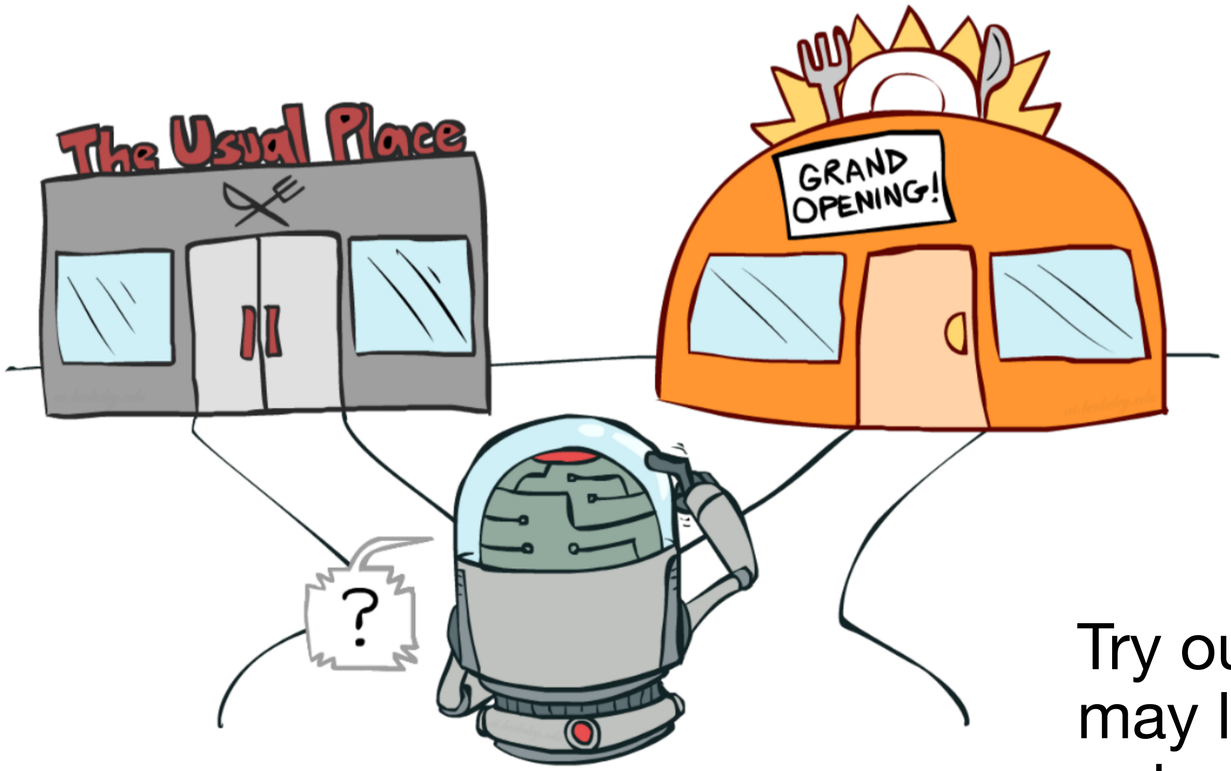
$$\alpha_k(s,a) = \frac{1}{n(s,a)}$$

A greedy policy w.r.t. $\hat{Q}$ doesn't usually satisfy the infinite visitation requirement

Stochastic policy $\mu(a \mid s)$ s.t.

$$\forall s, a : \mu(a \mid s) > 0:$$

Exploration-Exploitation Trade-off



Choose the best option based on current knowledge (which may be incomplete)

Try out new options that may lead to better outcomes in the future at the expense of an exploitation opportunity

Exploration vs Exploitation

- Greedy policy:
  $\pi(s) = \underset{a}{\text{argmax}}\, Q^*(s, a)$

Exploration vs Exploitation

- Greedy policy:
  $\pi(s) = \underset{a}{\text{argmax}}\, \hat{Q}(s, a)$
  - Quickly get some reward
  - Can get stuck in suboptimal action!

Example of Getting Stuck

Exploration vs Exploitation

- Uniform policy $\mu(a \mid s)$
  - Will *eventually* correctly estimate the $Q(s, a)$
  - May do extremely poorly in terms of rewards!
- Greedy policy: $\pi(s) = \underset{a}{\text{argmax}}\, \hat{Q}(s, a)$
  - Quickly get some reward
  - Can get stuck in suboptimal action!

Exploration vs Exploitation

- Uniform policy $\mu(a \mid s)$

- Greedy policy:
  $\pi(s) = \underset{a}{\text{argmax}}\, \hat{Q}(s, a)$

- ε-greedy policy:

$$\mu(\cdot \mid s) = \begin{cases} \text{select random action with probability } \varepsilon \\ \underset{a}{\text{argmax}}\, Q(s, a) \text{ with probability } 1 - \varepsilon \end{cases}$$

Exploration vs Exploitation

- Uniform policy $\mu(a \mid s)$

- Greedy policy:
  $\pi(s) = \underset{a}{\text{argmax}}\, \hat{Q}(s, a)$

- ε-greedy policy:

$$\mu(\cdot \mid s) = \begin{cases} \text{select random action with probability } \varepsilon \\ \underset{a}{\text{argmax}}\, Q(s, a) \text{ with probability } 1 - \varepsilon \end{cases}$$

- Boltzmann policy:

$$\mu(\cdot \mid s) = \underset{a}{\text{softmax}}\left(\frac{Q(s, a)}{\tau}\right)$$

Exploration vs Exploitation

- Uniform policy $\mu(a \mid s)$

- Greedy policy:
  $\pi(s) = \underset{a}{\text{argmax}}\, \hat{Q}(s, a)$

- ε-greedy policy:

$$\mu(\cdot \mid s) = \begin{cases} \text{select random action with probability } \varepsilon \\ \underset{a}{\text{argmax}}\, Q(s, a) \text{ with probability } 1 - \varepsilon \end{cases}$$

- Boltzmann policy:

$$\mu(\cdot \mid s) = \underset{a}{\text{softmax}}\left(\frac{Q(s, a)}{\tau}\right)$$

vs Exploitation

By Robbins-Monro theorem:

If ϵ_t satisfies Robbins-Monro conditions, then we converge $\hat{Q} \overset{a.s.}{\to} Q^*$

- Greedy policy:
  $\pi(s) = \underset{a}{\text{argmax}}\, \hat{Q}(s, a)$

- ε-greedy policy:

$$\mu(\cdot \mid s) = \begin{cases} \text{select random action with probability } \varepsilon \\ \underset{a}{\text{argmax}}\, Q(s, a) \text{ with probability } 1 - \varepsilon \end{cases}$$

- Boltzmann policy:

$$\mu(\cdot \mid s) = \underset{a}{\text{softmax}}\left(\frac{Q(s, a)}{\tau}\right)$$

Exploration vs Exploitation

- Uniform policy $\mu(a \mid s)$

- Simple to implement, usually performs quite well
- Might take some time to eliminate the suboptimal actions

- ε-greedy policy:

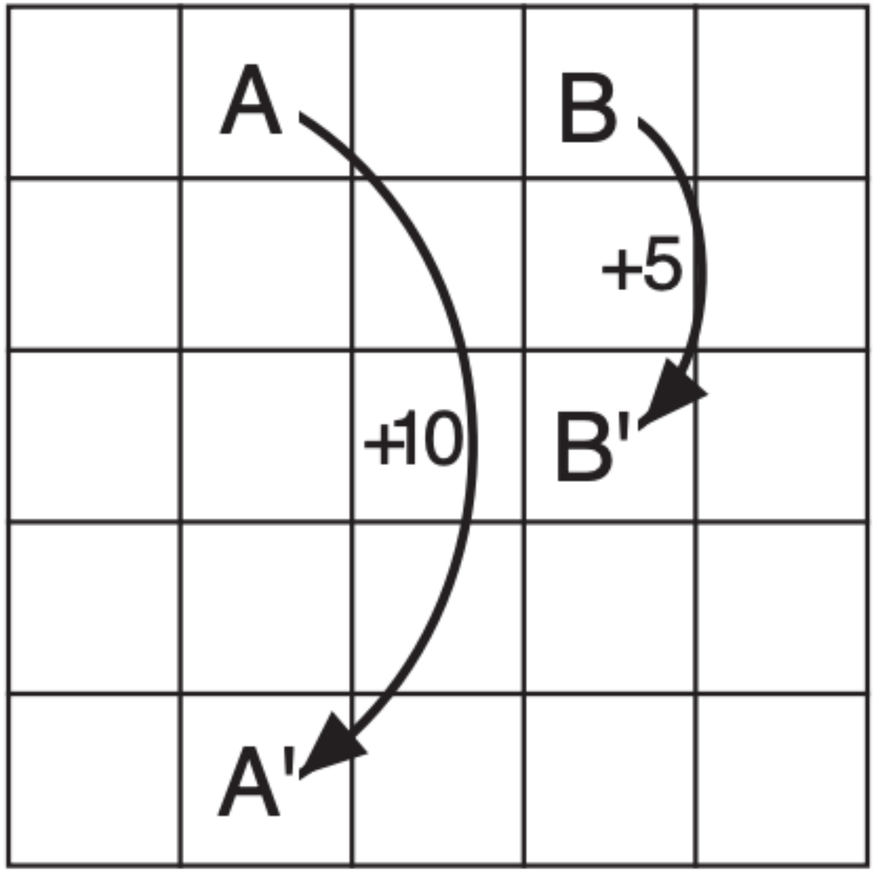
$$\mu(\cdot \mid s) = \begin{cases} \text{select random action with probability } \varepsilon \\ \underset{a}{\text{argmax}}\, Q(s,a) \text{ with probability } 1-\varepsilon \end{cases}$$

- Boltzmann policy:

$$\mu(\cdot \mid s) = \underset{a}{\text{softmax}}\left(\frac{Q(s,a)}{\tau}\right)$$

Example of Non-Infinite Visitation

Grid World Example (Sutton & Barto):



Page 72, Example 3.8: Gridworld

Model-Based Alternative: $R_{\max}$ algorithm

Optimizm in the face of uncertainty!

- **Idea**: Augment the initial MDP model for *implicit* exploration
- Create a virtual "fairy tale" state s^*, initialize $P(s^* \mid s, a) = 1$ and $\hat{r}(s, a) = R_{\max}$
- Compute optimal policy π_t w.r.t. to the augmented model $\hat{P}(s' \mid s, a)$ and $\hat{r}(s, a)$
- Play out π_t in the true environment
- Update $\hat{P}(s' \mid s, a)$ and $\hat{r}(s, a)$ in our model
- If observed "enough" transitions / rewards, recompute policy π_{t+1} according to current model $\hat{P}(s' \mid s, a)$ and $\hat{r}(s, a)$

Requirements Differ

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$$

$$s' \sim p(\,.\,|\,s,a)$$

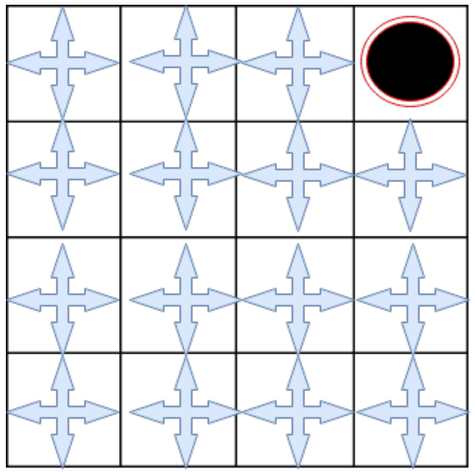
$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)(r + \gamma Q_k(s',a') - Q_k(s,a))$$

$$s' \sim p(\,.\,|\,s,a), a' \sim \pi(\,.\,|\,s)$$

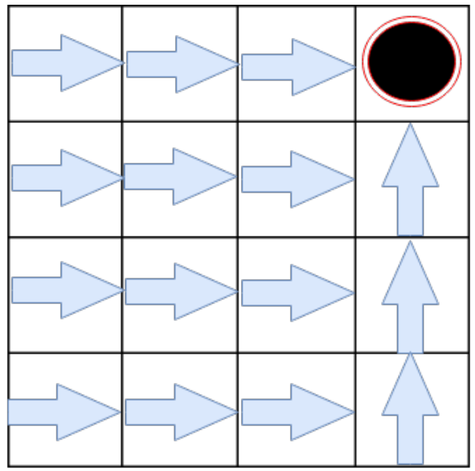
On-policy vs Off-Policy

On-policy learning

- Learn about behaviour policy μ from experience sampled from μ



Behavior Policy



Target Policy

Off-policy learning

- Learn about target policy π from experience sampled from μ
- Learn from observing humans or other agents (e.g., from logged data)
- Learn about multiple policies while following one policy
- Learn about greedy policy while following exploratory policy
- Reuse experience from old policies (e.g., from your own past experience)

Q-Learning

- Parameters: ε, α
- Initialize $Q_0(s,a)\ \forall s,a$
- For $k = 0,1,\ldots$

1. $a \sim \mu(\,.\,|\,s) = \begin{cases} \text{select random action with probability } \varepsilon \\ argmax_a Q_k(s,a) \text{ with probability } 1-\varepsilon \end{cases}$
2. Observe r and $s' \sim p(\,.\,|\,s,a)$
3. $Q_{k+1}(s,a) = Q_k(s,a) + \alpha(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$

Q-Learning

- Parameters: ε, α
- Initialize $Q_0(s,a)\ \forall s,a$
- For $k = 0,1,\ldots$

1. $a \sim \mu(\,.\,|\,s) = \begin{cases} \text{select random action with probability } \varepsilon \\ argmax_a Q_k(s,a) \text{ with probability } 1-\varepsilon \end{cases}$
2. Observe r and $s' \sim p(\,.\,|\,s,a)$
3. $Q_{k+1}(s,a) = Q_k(s,a) + \alpha(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$

Q-Learning learns Q^* using samples from another policy!

Q-Learning

- Parameters: ε, α
- Initialize $Q_0(s,a)\ \forall s,a$
- For $k = 0,1,\ldots$

1. $a \sim \mu(\,.\,|\,s) = \begin{cases} \text{select random action with probability } \varepsilon \\ argmax_a Q_k(s,a) \text{ with probability } 1-\varepsilon \end{cases}$
2. Observe r and $s' \sim p(\,.\,|\,s,a)$
3. $Q_{k+1}(s,a) = Q_k(s,a) + \alpha(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$

E.g., π^{expert}

Q-Learning learns Q^* using samples from another policy!

Q-Learning

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$$

Theorem:

If α_k satisfies Robbins-Monro conditions $\sum_k \alpha_k = +\infty$ and $\sum_k \alpha_k^2 < +\infty$,

and we visit all state-action pairs infinitely often,

then $Q_k \overset{a.s.}{\to} Q^*$

Q-Learning

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$$

Theorem:

If α_k satisfies Robbins-Monro conditions $\sum_k \alpha_k = +\infty$ and $\sum_k \alpha_k^2 < +\infty$,

and we visit all state-action pairs infinitely often,

then $Q_k \overset{a.s.}{\to} Q^*$

- Memory required: $\mathcal{O}(|\mathcal{S}||\mathcal{A}|)$
- Computational time: table lookup + argmax over actions $\mathcal{O}(|\mathcal{A}|)$

SARSA

- Parameters: ε, α
- Initialise $Q_0(s,a)\ \forall s,a$
- For $k = 0,1,\ldots$

1. $a \sim \mu(\,.\,|\,s) = \begin{cases} \text{select random action with probability } \varepsilon \\ argmax_a Q_k(s,a) \text{ with probability } 1-\varepsilon \end{cases}$
2. Observe r and $s' \sim p(\,.\,|\,s,a)$
3. $a' \sim \mu(\,.\,|\,s') = \begin{cases} \text{select random action with probability } \varepsilon \\ argmax_a Q_k(s',a) \text{ with probability } 1-\varepsilon \end{cases}$
4. $Q_{k+1}(s,a) = Q_k(s,a) + \alpha(r + \gamma Q_k(s',a') - Q_k(s,a))$

Q-Learning vs SARSA

Q-Learning: $Q_{k+1}(s,a) = Q_k(s,a) + \alpha(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$

SARSA: $Q_{k+1}(s,a) = Q_k(s,a) + \alpha(r + \gamma Q_k(s',a') - Q_k(s,a))$

Q-Learning vs SARSA

Q-Learning: $Q_{k+1}(s,a) = Q_k(s,a) + \alpha(r + \gamma \max_{a'} Q_k(s',a') - Q_k(s,a))$

SARSA: $Q_{k+1}(s,a) = Q_k(s,a) + \alpha(r + \gamma Q_k(s',a') - Q_k(s,a))$

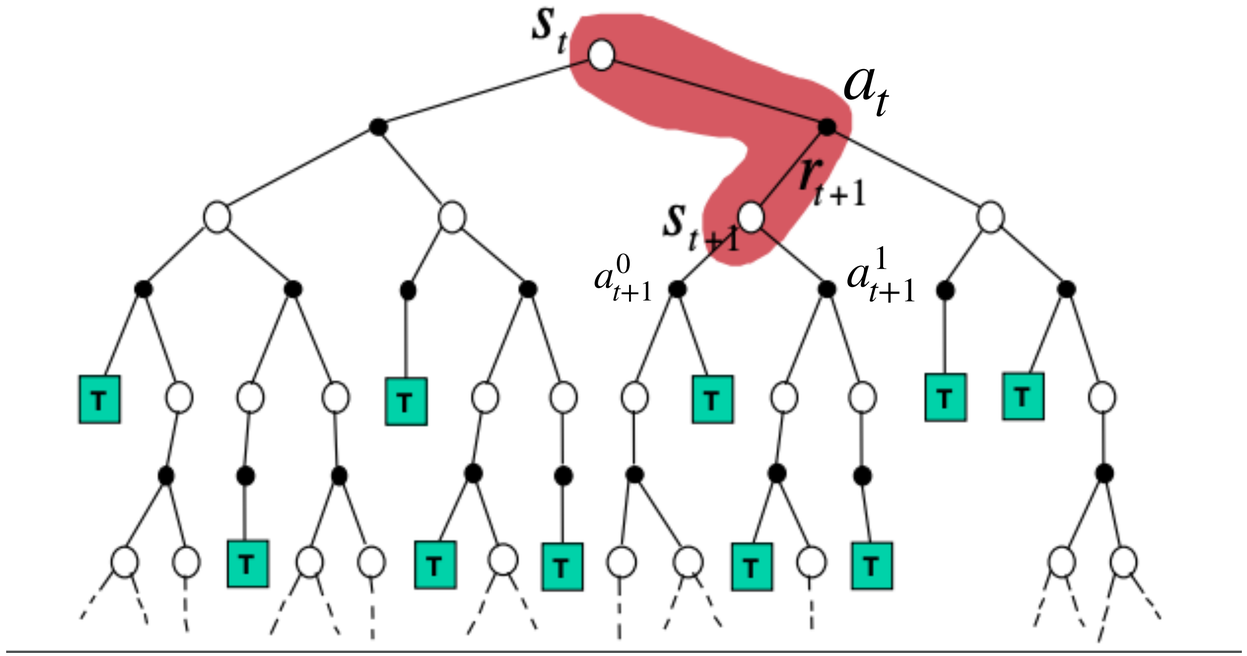
Q-Learning: Learns Q^{π_G}

SARSA: Learns Q^{μ}, where $\mu = \epsilon$-greedy(Q_k)

Expected SARSA

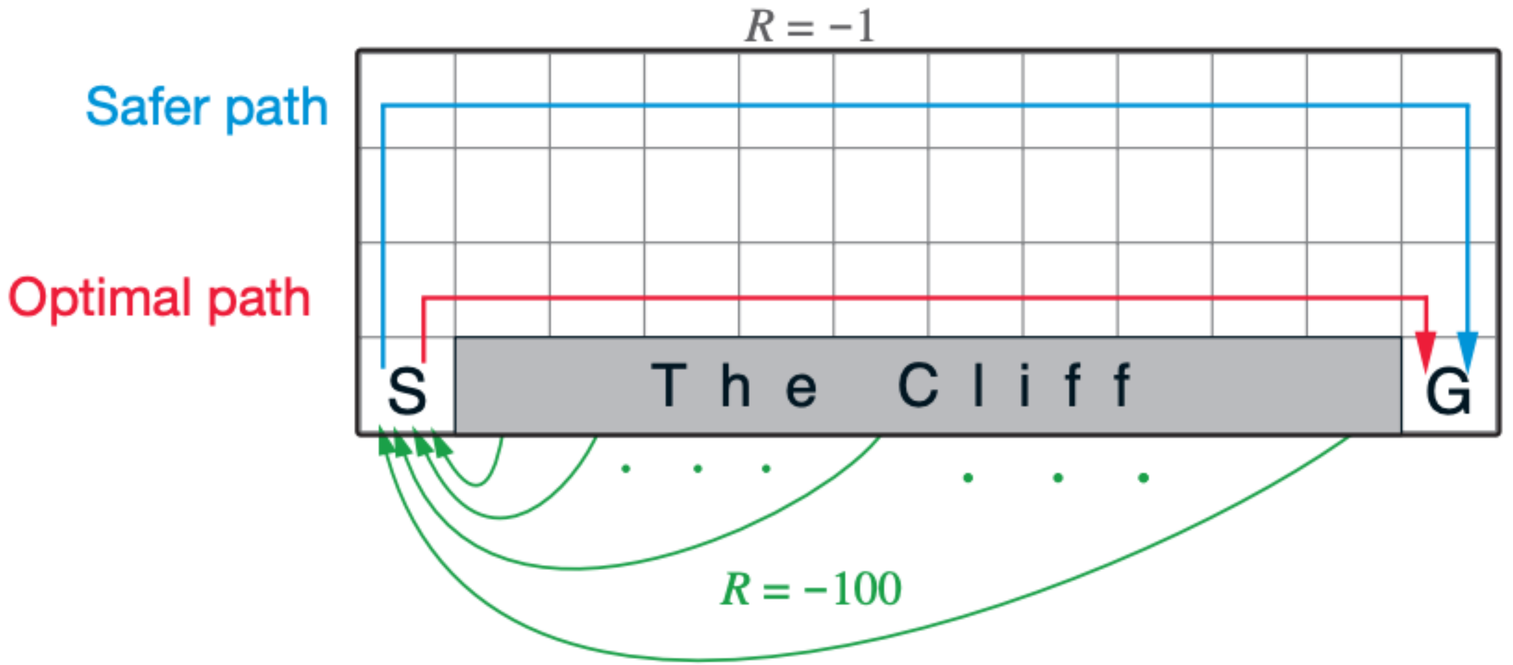
$$Q_{k+1}(s,a) \leftarrow Q_k(s,a) + \alpha(r + \gamma \mathbb{E}_{a' \sim \mu_k(.|s')} Q_k(s',a') - Q_k(s,a))$$

$$\mu_k(.\,|\,s') = \begin{cases} \text{select random action with probability } \varepsilon \\ \underset{a}{\text{argmax}}\, Q_k(s',a) \text{ with probability } 1-\varepsilon \end{cases}$$



Example: Cliff Walking

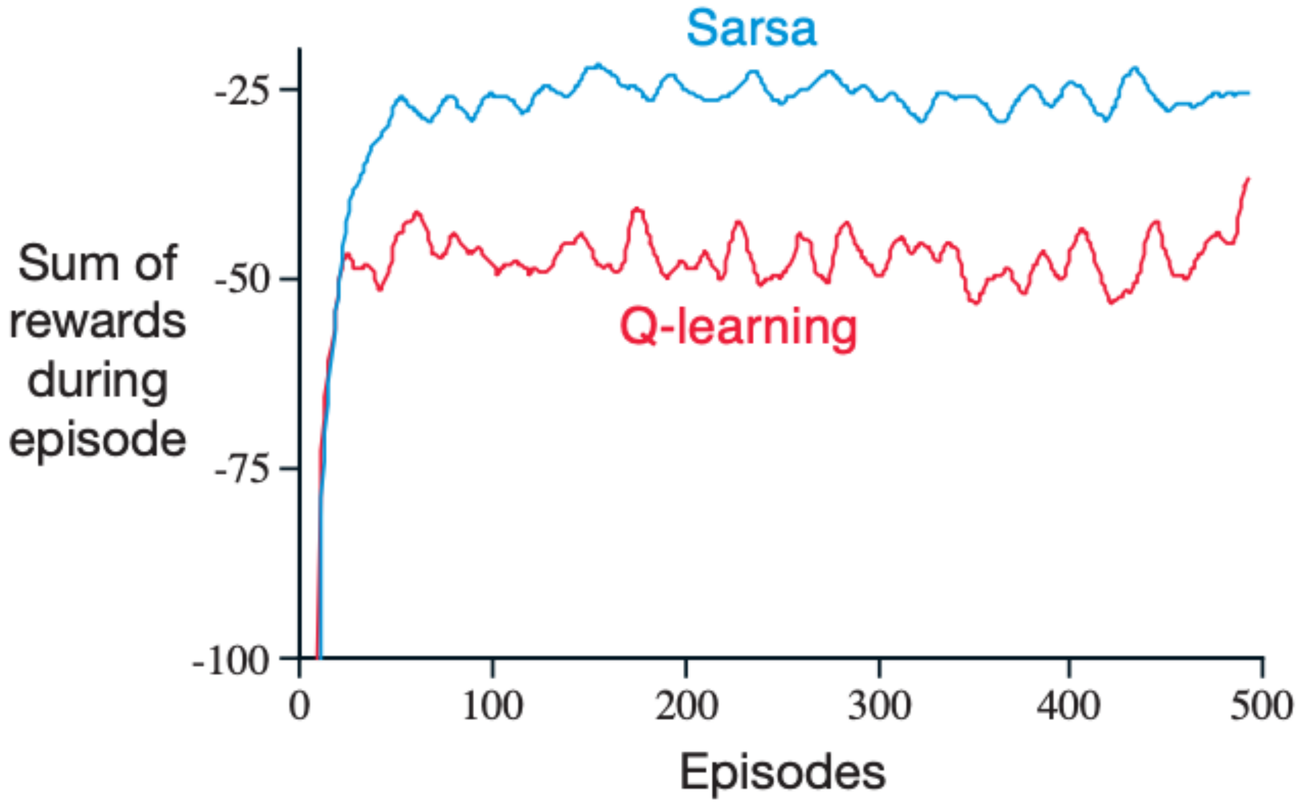
$\gamma = 1, \varepsilon = 0.1$. Agent gets -1 for each step.



Source

Which policy is learned by Q-learning and SARSA?

Example: Cliff Walking



Source

Of course, if ε were gradually reduced, both methods would asymptotically converge to the optimal policy.

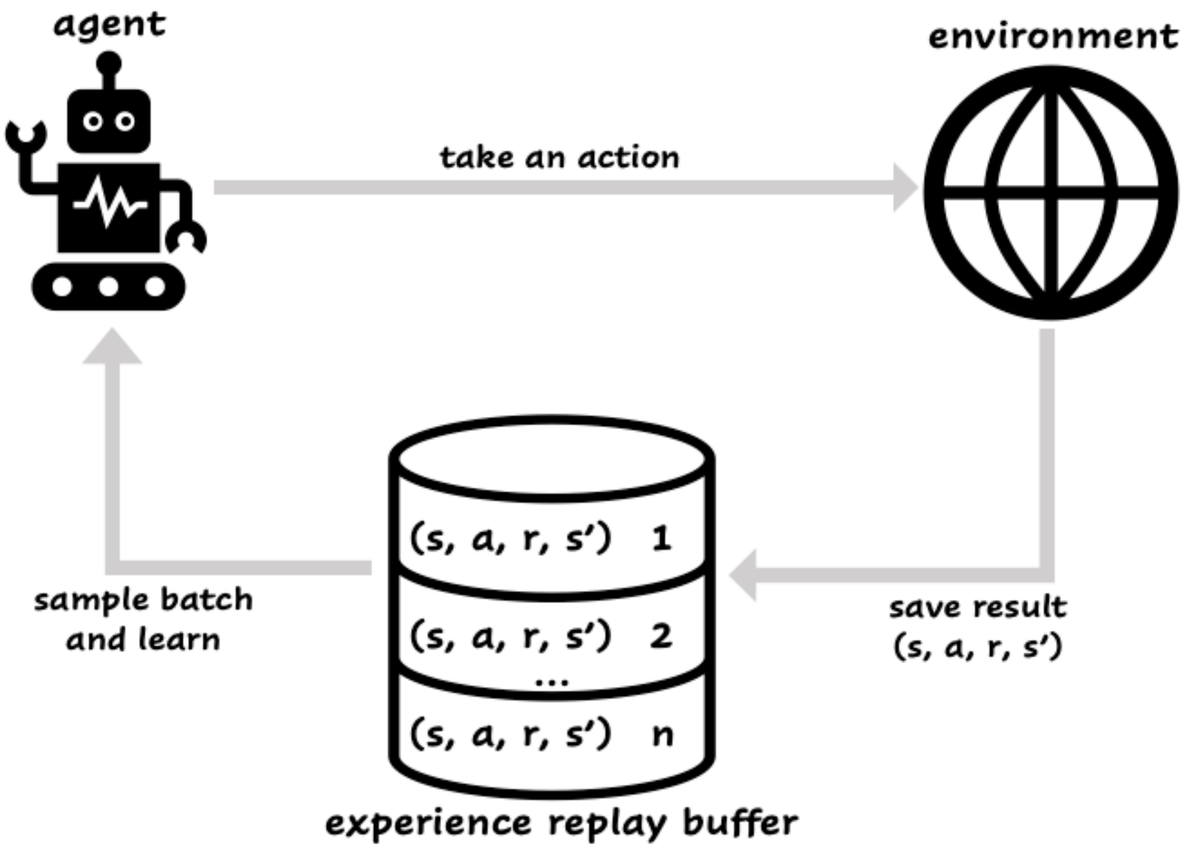
Experience Replay Buffer

Advantages:

- No need to revisit same states many times
- Make the estimators consistent with the current policy, update estimators
- Decorrelate update samples to maintain i.i.d. assumption

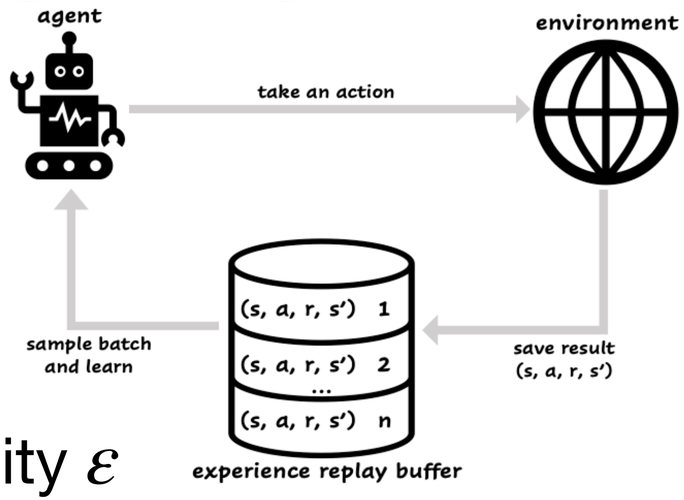
Disadvantages:

- Not applicable for the on-policy learning



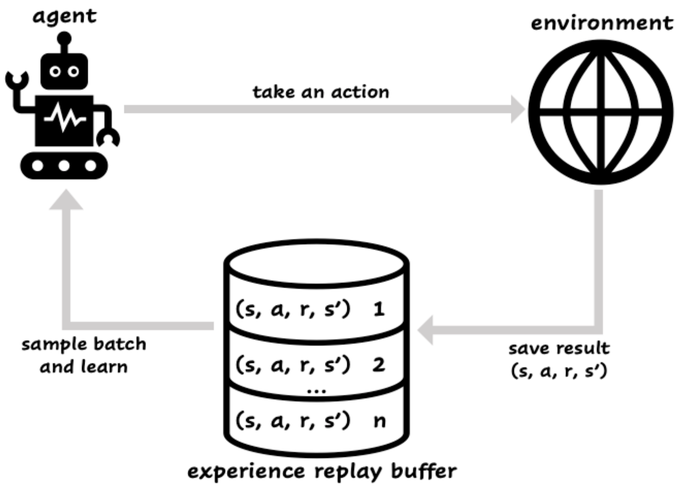
Q-Learning With RB: Step By Step

- Parameters: ε, α
- Initialize $Q_0(s, a)\ \forall s, a$
- For $k = 0,1,\ldots$



1. $a \sim \mu(\,.\,|\,s) = \begin{cases} \text{select random action with probability } \varepsilon \\ argmax_a Q_k(s, a) \text{ with probability } 1 - \varepsilon \end{cases}$
2. Observe r and $s' \sim p(\,.\,|\,s, a)$
3. Store into buffer (s, a, r, s')
4. Sample from buffer $(\tilde{s}, \tilde{a}, \tilde{r}, \tilde{s}')$
5. $Q_{k+1}(\tilde{s}, \tilde{a}) = Q_k(\tilde{s}, \tilde{a}) + \alpha(r + \gamma \max_{a'} Q_k(\tilde{s}', a') - Q_k(\tilde{s}, \tilde{a}))$

SARSA With RB?



Optimistic Q-Learning

Optimizm in the face of uncertainty!

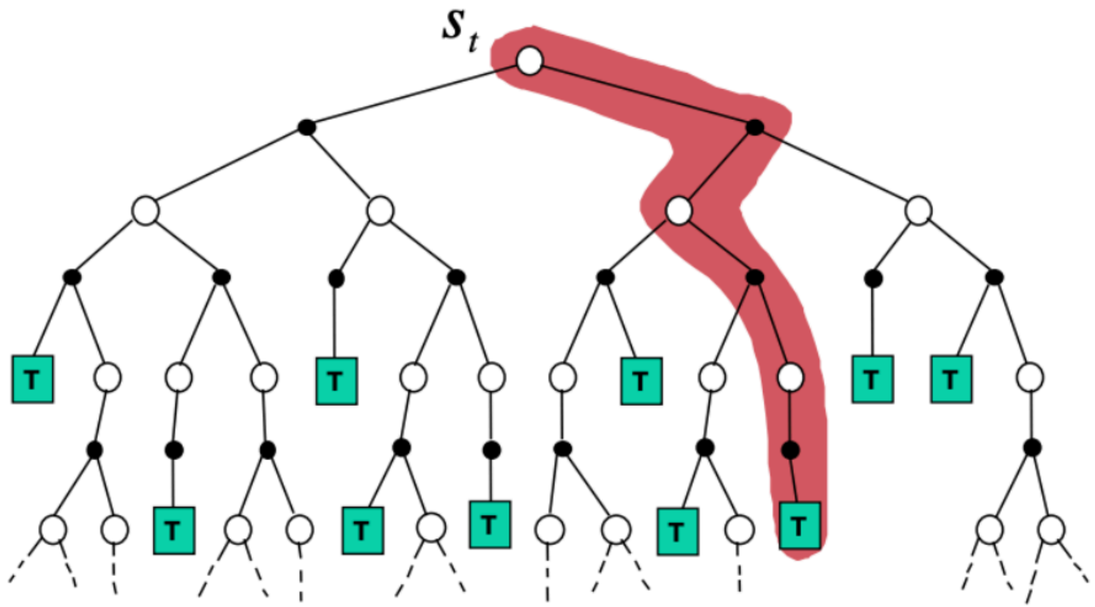
- **Idea**: Similar to $R_{\max}$ → Initialize Q_0 for *implicit* exploration
- Initialize $Q_0 = \frac{R_{\max}}{1-\gamma} \Pi_{t=1}^{T_{init}} (1-\alpha_t)^{-1}$
- Act greedily in the environment and proceed with Q-Learning

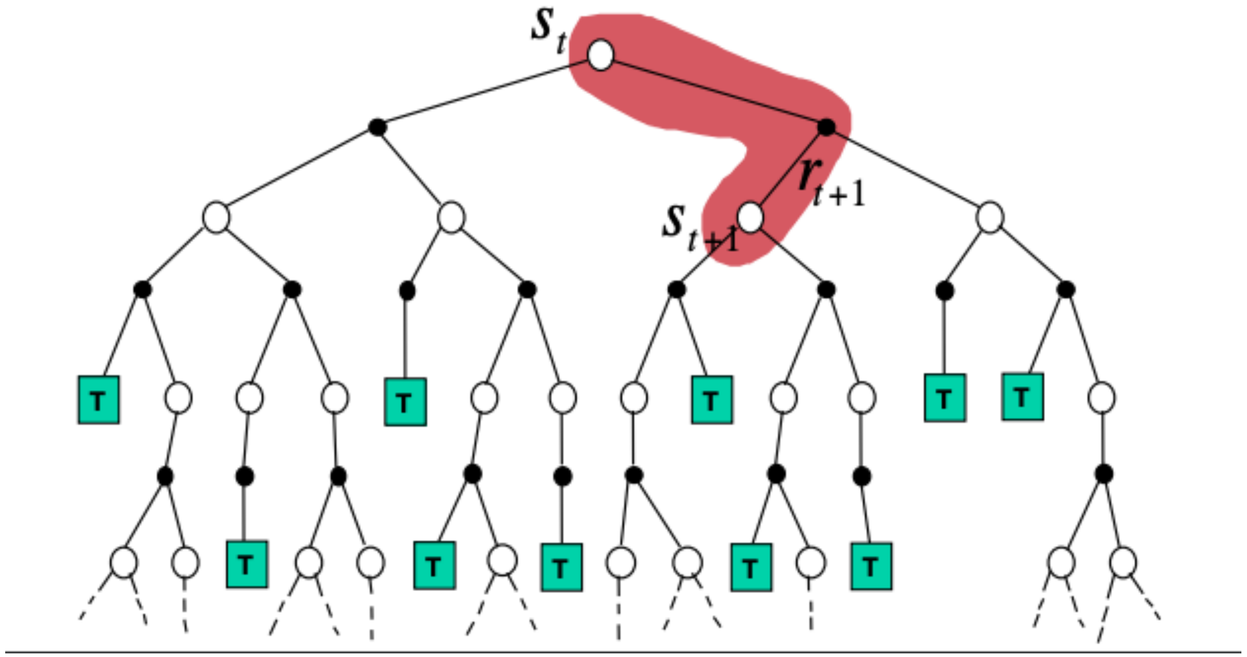
Theorem:

With probability $1-\delta$, optimistic Q-Learning **obtains an ϵ-optimal policy** after a number of time steps that is polynomial in $|\mathcal{S}|$, $|\mathcal{A}|$, $1/\epsilon$ and $\log(1/\delta)$

Bias-Variance Trade-off

$$Q(s,a) \leftarrow Q(s,a) + \alpha(y_Q - Q(s,a))$$





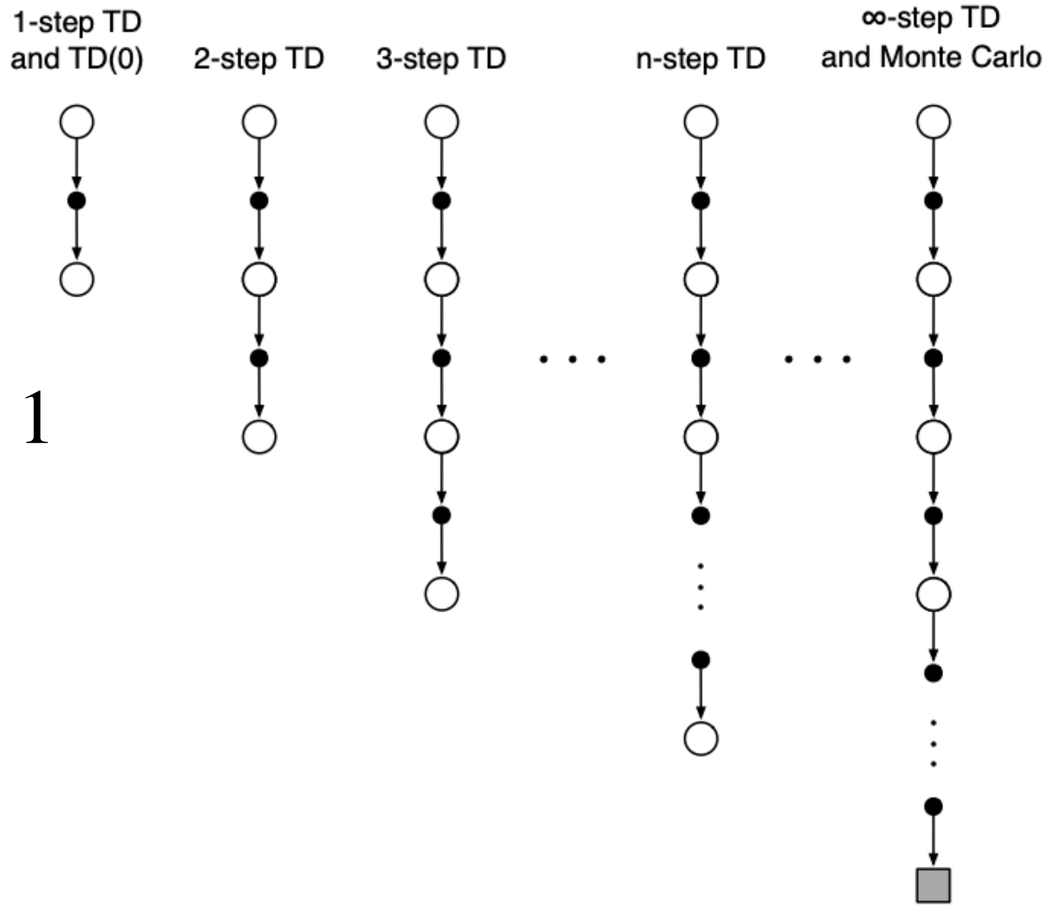
N-step SARSA

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha(y_Q^N - Q_k(s,a))$$

$$y_Q^N = r + \gamma r' + \gamma^2 r'' + \ldots + \gamma^N Q_k(s^{(N)}, a^{(N)})$$

$$s^{(n)} \sim p(\,.\,|\,s^{(n-1)}, a^{(n-1)}),\ a^{(n)} \sim \mu(a\,|\,s),\ n \geq 1$$

Ranging N we can control the trade-off between bias and variance.



Credit Assignment

At the time step t:

$$\delta_t^1 = r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$$

$$\delta_t^2 = r_t + \gamma r_{t+1} + \gamma^2 Q(s_{t+2}, a_{t+2}) - Q(s_t, a_t)$$

$$\delta_t^N = r_t + \gamma r_{t+1} + \ldots + \gamma^N Q(s_{t+N}, a_{t+N}) - Q(s_t, a_t)$$

$$\delta_t^N = \sum_{n=0}^{N-1} \gamma^n \delta_{t+n}^1$$

Credit Assignment

At the time step t:

$$\delta_t^1 = r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$$

$$\delta_t^2 = r_t + \gamma r_{t+1} + \gamma^2 Q(s_{t+2}, a_{t+2}) - Q(s_t, a_t)$$

$$\delta_t^N = r_t + \gamma r_{t+1} + \ldots + \gamma^N Q(s_{t+N}, a_{t+N}) - Q(s_t, a_t)$$

$$\delta_t^N = \sum_{n=0}^{N-1} \gamma^n \delta_{t+n}^1$$

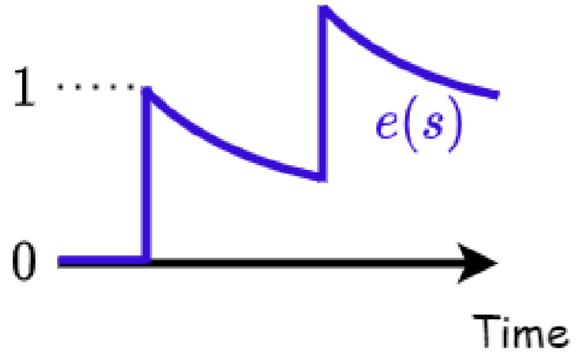
N-step SARSA: $Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t^N$

1. $Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t^1$
2. $Q(s, a) \leftarrow Q(s, a) + \alpha \gamma \delta_{t+1}^1$
3. ....

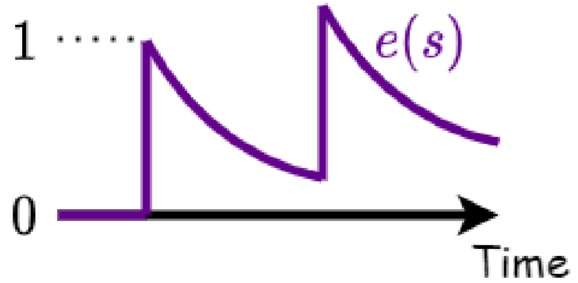
Eligibility Traces

$$e_0(s,a) = 0$$

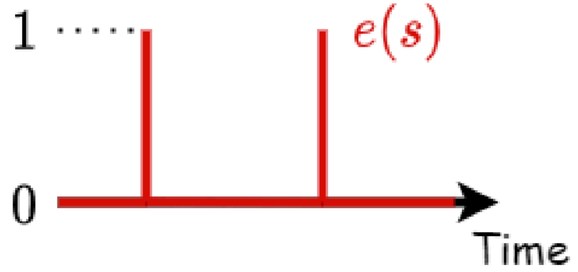
$$e_t(s,a) = \gamma\lambda e_{t-1}(s,a) + \mathbb{I}[S_t = s, A_t = a]$$



$\lambda = 1$



$0 < \lambda < 1$



$\lambda = 0$

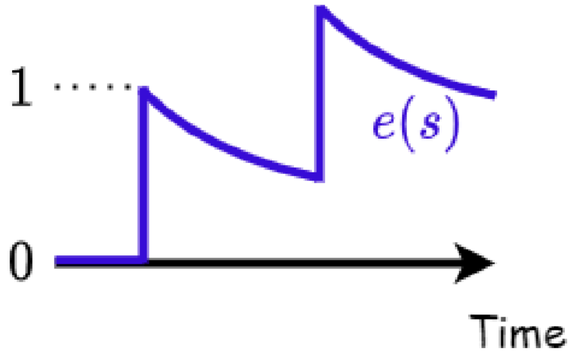
SARSA(λ)

$$e_0(s,a) = 0$$

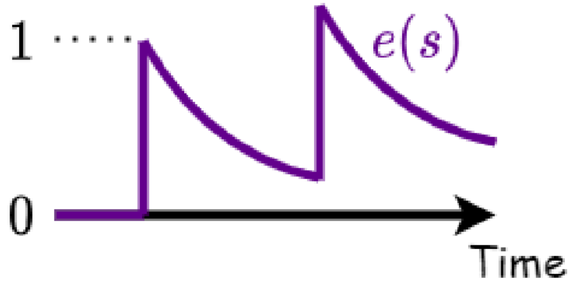
$$e_t(s,a) = \gamma\lambda e_{t-1}(s,a) + \mathbb{I}[S_t = s, A_t = a]$$

1. $Q(s,a) \leftarrow Q(s,a) + \alpha e_1(s,a)\delta_0^1$
2. $Q(s,a) \leftarrow Q(s,a) + \alpha e_2(s,a)\delta_1^1$
3. ....

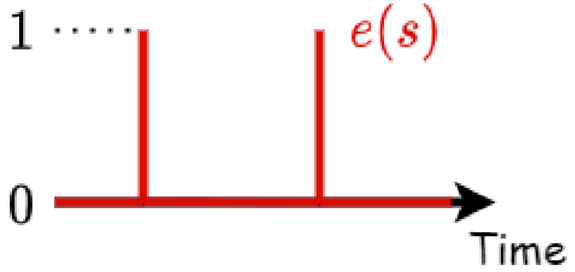
$$Q(s,a) \leftarrow Q(s,a) + \alpha \sum_{t \geq 0} (\gamma\lambda)^t \delta_t^1$$



$\lambda = 1$



$0 < \lambda < 1$



$\lambda = 0$

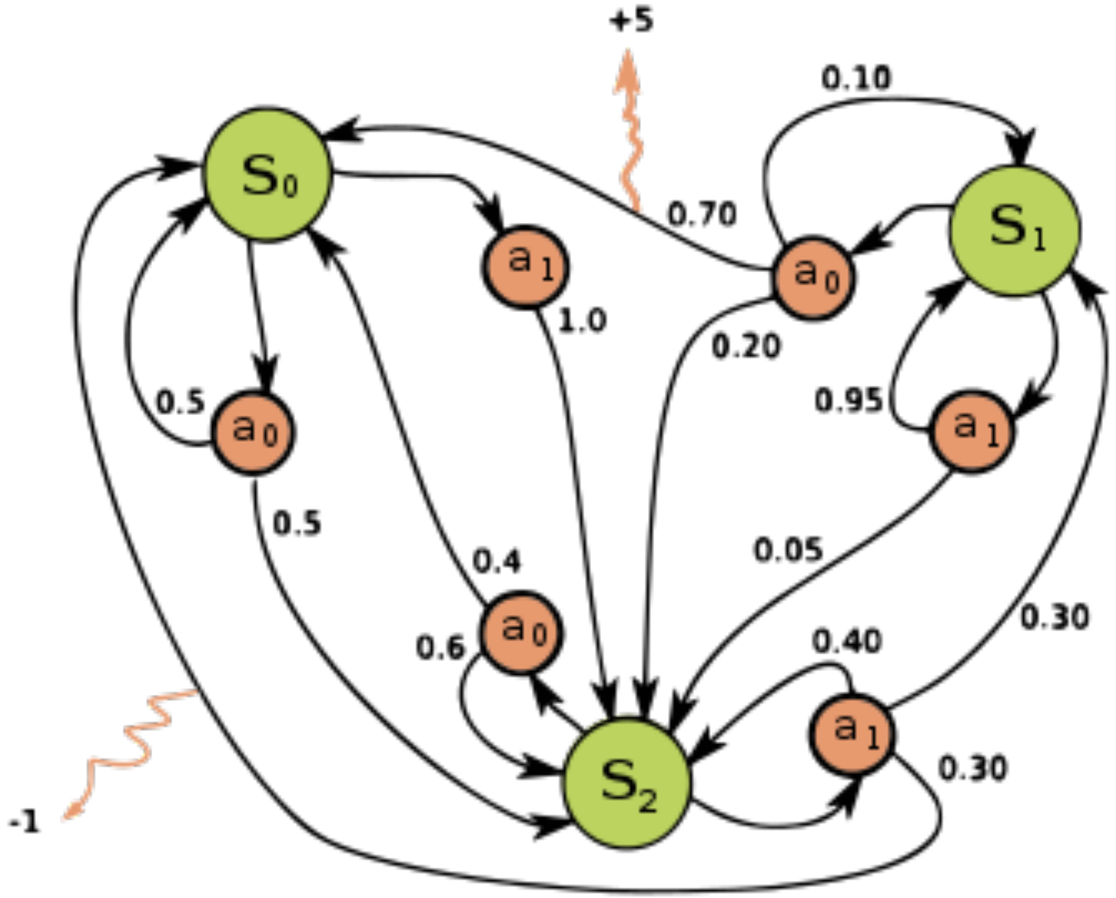
SARSA(λ)

$$\sum_{t \geq 0} (\gamma\lambda)^t \delta_t^1 = (1-\lambda) \sum_{N \geq 1} \lambda^{N-1} \delta_0^N$$

We ensemble all N-step updates.

Next Challenge — Scaling Up

1. $p(s' \mid s, a)$ is fully known
2. $|\mathcal{S}| < \infty$ → Lecture 3
3. $|\mathcal{A}| < \infty$ → Lecture 4



Background

1. Reinforcement Learning Textbook (in Russian): 3.4 - 3.5
2. Sutton & Barto, Chapter 5 + 6 + 7*
3. Practical RL course by YSDA, week 3
4. DeepMind course, lectures 5 + 6

Thank you for your attention!

Appendix: Reward Augmentation

As we have discussed in the Lecture 1, one can design an RL task with reward function as $r(s, a)$ or $r(s, a, s')$. Let us show that **these two setting are mathematically equivalent in a sense of producing the same optimal policies**

Appendix: Reward Augmentation

As we have discussed in the Lecture 1, one can design an RL task with reward function as $r(s, a)$ or $r(s, a, s')$. Let us show that **these two setting are mathematically equivalent in a sense of producing the same optimal policies**

- Let's assume a discount factor $0 < \gamma \leq 1$ (for $\gamma = 1$ do not rescale the rewards)
- At first, let's look at the MDPs with $|\mathcal{S}| < \infty$ and $|\mathcal{A}| < \infty$ (*can be relaxed with the proof via conditional expectation*)
- We can produce an auxiliary MDP for the $r(s, a, s')$ to get $r(s, a)$ in order to apply the same formalizm as we do in this course

Appendix: Reward Augmentation

- Start from $M = (S, A, P, \gamma, R)$, where $R = r(s, a, s')$
- Build $M' = (S', A', P', \gamma, R')$ as following:

    1. $S' = S \cup \{aux(s, a, s') : s \in S, a \in A, s' \in S, P(s' \mid s, a) > 0\}$, where $aux(s, a, s')$ are the new auxiliary states that absorb the reward dependence on the transition outcome.
    2. Introduce in auxiliary states dummy action a^{dummy}, which is the only available action.
    3. Create new transitions from the auxiliary states $P'(aux(s, a, s') \mid s, a) = P(s' \mid s, a)$.
    4. Force certainty in transitions to the new state from the absorbing auxiliary one $P(s' \mid aux(s, a, s'), a^{dummy}) = 1$
    5. Augment the rewards to be $R'(s, a) = 0 \ \forall s \in S$ and $R'(aux(s, a, s'), a^{dummy}) = \gamma^{-1} R(s, a, s')$

Interpretation:
we make a step with initial probability to auxiliary state with initial transition probability, collect the reward by taking the only available action and exit to s'

Appendix: Reward Augmentation

Why $M = (S, A, P, \gamma, R) \to M' = (S', A', P', \gamma, R')$ works?

- Consider a policy π on M and π' on M', where the latter uses initial π on the original states $s \in S$ and chooses the only available action a^{dummy} on every auxiliary one.

- Then for a trajectory $(s_0, a_0, s_1, a_1, s_2, \ldots)$ in initial MDP M we get a new trajectory $(s_0, a_0, aux(s_0, a_0, s_1), a^{dummy}, s_1, a_1, aux(s_1, a_1, s_2), a^{dummy}, s_2, \ldots)$ in the augmented MDP M'

- The returns are the following:

$$M \implies \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1})$$

Because auxiliary state is always at the next timestamp from the initial MDP's t

$$M' \implies \sum_{t=0}^{\infty} \gamma^t R'(s_t, a_t) + \sum_{t=0}^{\infty} \gamma^{t+1} R'(aux(s_t, a_t, s_{t+1}), a^{dummy}) = 0 + \sum_{t=0}^{\infty} \gamma^{t+1} \gamma^{-1} R(s_t, a_t)$$

- This implies that our **returns are equivalent everywhere**

- Therefore, $V^{\pi'}_{M'}(s) = V^{\pi}_{M}(s)\ \forall s \in S$, which **implies that an optimal V^* induces the same π_G**

Appendix: Reward Augmentation Example

Imagine we have a business and we are in the fixed initial state s.

We can take some risky action a that either with probability $1-p$ pays off $r(s,a,s') = +100$, if we end up in the terminal state s' = business is booming.

Alternatively, with probability p the same action can lead to another terminal state with $r(s,a,s'') = -1{,}000$, if we end up in the state s'' = business has defaulted.

In both these states the game ends.

Appendix: Reward Augmentation Example

Imagine we have a business and we are in the fixed initial state s.

We can take some risky action a that either with probability $1-p$ pays off $r(s,a,s') = +100$, if we end up in the terminal state s' = business is booming.

Alternatively, with probability p the same action can lead to another terminal state with $r(s,a,s'') = -1{,}000$, if we end up in the state s'' = business has defaulted.

In both these states the game ends.

Solution (1/2):

Add auxiliary state that encodes the outcome of taking this risky action.

Let $u_+ := aux(s,a,s'), u_- := aux(s,a,s'')$, new state space $S' = \{s, s', s''\} \cup \{u_+, u_-\}$, new action space $A' = \{a, a^{dummy}\}$.

Augment the transitions $P'(u_+ \mid s,a) = 1-p$, $P'(u_- \mid s,a) = p$, $P'(s' \mid u_+, a^{dummy}) = 1$, $P'(s'' \mid u_-, a^{dummy}) = 1$

Augment the reward function $R'(s,a) = 0$, $R'(u_+, a^{dummy}) = \dfrac{+100}{\gamma}$, $R'(u_-, a^{dummy}) = \dfrac{-1{,}000}{\gamma}$.

Appendix: Reward Augmentation Example

Imagine we have a business and we are in the fixed initial state s.

We can take some risky action a that either with probability $1-p$ pays off $r(s,a,s') = +100$, if we end up in the terminal state s' = business is booming.

Alternatively, with probability p the same action can lead to another terminal state with $r(s,a,s'') = -1{,}000$, if we end up in the state s'' = business has defaulted.

In both these states the game ends.

Solution (2/2):

We can see that the Value functions yield the same outcomes, meaning that π_G, induced by V^* will be equivalent in these two MDPs.

$$V_M^{\pi} = (1-p) \cdot 100 + p \cdot -1{,}000$$

$$V_{M'}^{\pi'} = 0 + \gamma((1-p) \cdot \frac{100}{\gamma} + p \cdot \frac{-1{,}000}{\gamma})$$

Appendix: Reward Augmentation

The same applies to continuous spaces

- Use conditional expectation:

$$r(s,a) = \mathbb{E}_{s' \sim p(\cdot|s,a)}[R(s,a,s')] = \int_S R(s,a,s')p(ds' \,|\, s,a)$$

- Note that value functions still coincide (by Tower property, applied by conditioning on (s_t, a_t)):

$$V^\pi(s) = \mathbb{E}[\sum_{t=0}^{\infty} R(s_t,a_t,s_{t+1}) \,|\, s_0 = s] = \mathbb{E}[\sum_{t=0}^{\infty} \mathbb{E}[R(s_t,a_t,s_{t+1}) \,|\, s_t,a_t] \,|\, s_0 = s] = \mathbb{E}[\sum_{t=0}^{\infty} r(s_t,a_t) \,|\, s_0 = s]$$

- Then the Bellman operator remains unchanged and the same argument goes through

$$TV = \sup_{a \in A}\{r(s,a) + \gamma \int V(s') \; p(ds' \,|\, s,a)\}$$